
Refprop

Release 11.0.0-a.1

Matthew Boyd and Ian Bell

Sep 08, 2026

CONTENTS:

1	Introduction	1
1.1	Disclaimer	1
1.2	Contact	1
1.3	Overview	1
2	User's Guide	3
2.1	Graphical User Interface	3
3	API Reference	5
4	Indices and tables	91
	Index	93

INTRODUCTION

Refprop is a software package developed by the National Institute of Standards and Technology (NIST) that provides comprehensive thermodynamic and transport property data for a variety of industrially important pure fluids and mixtures. Refprop is commonly utilized in the fields of engineering, thermodynamics, and process simulation. The current version of Refprop, Version 11, can be accessed here: <https://www.nist.gov/srd/refprop>.

1.1 Disclaimer

This software is an official contribution of the National Institute of Standards and Technology (NIST). NIST assumes no responsibility whatsoever for its use by other parties, and makes no guarantees, expressed or implied, about its quality, reliability or any other characteristic.

1.2 Contact

For questions or comments about this software, please contact:

Matthew Boyd
matthew.boyd@nist.gov

Ian Bell
ian.bell@nist.gov

Applied Chemicals and Materials Division
Material Measurement Laboratory
National Institute of Standards and Technology
Boulder, CO 80305

1.3 Overview

Lorem ipsum

Lorum ipsum

2.1 Graphical User Interface

Lorum ipsum

API REFERENCE

Typedefs

using **Handle** = uint64_t

class **IRefprop**

#include <refprop.h> Subclassed by ThermoPhysModel

Public Functions

virtual ~**IRefprop**() = default

virtual std::string **name**() const = 0

virtual std::string **cas**() const = 0

class **RefpropManager**

#include <refprop.h>

Public Types

using **Handle** = uint64_t

Public Static Functions

static *Handle* **create_model**(const std::string &fluid_names, const std::string &mix_coef_file, const std::string &ref_state, const bool is_predefined_mix, const std::string &ideal_gas_model_params, const std::string &residual_model_params)

static std::shared_ptr<const *IRefprop*> **get_model**(const *Handle* handle)

namespace **rp**

Functions

void **A_ideal_gas**(const *Handle* model_hdl, const int i, const int j, const double T, const double D, const std::vector<double> &z, double &A_ig)

Compute the ideal-gas part of the reduced Helmholtz energy or its derivatives as functions of temperature and density for a mixture.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **i** – [const int, in] :: Order of the temperature derivative
- **j** – [const int, in] :: Order of the density derivative
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **A_ig** – [double&, out] :: Ideal gas part of the reduced Helmholtz energy derivative

void **A_residual**(const *Handle* model_hdl, const int i, const int j, const double T, const double D, const std::vector<double> &z, double &A_r)

Compute the residual part of the reduced Helmholtz energy or its derivatives as functions of temperature and density for a mixture.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **i** – [const int, in] :: Order of the temperature derivative
- **j** – [const int, in] :: Order of the density derivative
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **A_r** – [double&, out] :: Residual part of the reduced Helmholtz energy derivative

void **AGcpp**(const *Handle* model_hdl, const double T, const double D, const std::vector<double> &z, double &a, double &g)

Compute Helmholtz and Gibbs energies as functions of temperature, density, and composition. These are not residual values (those are calculated by GIBBS). See warning in subroutines THERM or ALLPROPS.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]

- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **a** – [double&, out] :: Helmholtz energy [J/mol]
- **g** – [double&, out] :: Gibbs free energy [J/mol]

void **B12cpp**(const *Handle* model_hdl, const double T, const std::vector<double> &z, double &B)

Compute B12 as a function of temperature and composition for a binary mixture.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **B** – [double&, out] :: B12 [L/mol]

void **BLCRVcpp**(const *Handle* model_hdl, const double D, const std::vector<double> &z, double &T, int &ierr, std::string &herr, const int herr_length = 255)

Calculate the temperature along the Boyle curve for the input density.

This line starts at zero density at the temperature where B=0, and passes into the liquid phase without crossing the two-phase. It ends at a saturated liquid state very close to the critical point. The argument z in this routine is an array with the mole fractions of the mixture. If the input T is non-zero, it is used as the initial guess.

Flags flags

0 Successful **151** Iteration failed to converge

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **D** – [const double, in] :: Density [mol/l]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **T** – [double&, in/out] :: Temperature [K]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **CHEMPOTcpp**(const *Handle* model_hdl, const double T, const double D, const std::vector<double> &z, std::vector<double> &u, int &ierr, std::string &herr, const int herr_length = 255)

Compute the chemical potentials for each of the nc components of a mixture.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **u** – [const std::vector<double>&, in] :: Vector of the chemical potentials [J/mol]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: Error string length

void **CP0cpp**(const *Handle* model_hdl, const double T, const std::vector<double> &z, double &Cp)

Calculate Cp0 for a mixture given temperature and composition.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **Cp** – [double&, out] :: Isobaric heat capacity [J/mol-K]

void **CRITPcpp**(const *Handle* model_hdl, const std::vector<double> &z, double &Tc, double &Pc, double &Dc, int &ierr, std::string &herr, const int herr_length = 255)

Calculate critical parameters as a function of composition.

The critical parameters are estimates based on polynomial fits to the binary critical lines. For 3 or more components, combining rules are applied to the constituent binaries.

If SATSPLN has been called and the input composition sent here is the same as that sent to SATSPLN, the values calculated from the splines are returned, which are nearly exact. During the call to SATSPLN, the true critical point, maximum pressure point, and maximum temperature point along the saturation lines are determined. Without the splines and for a system with three or more components, the values from this routine are only rough estimates.

Flags flags

0 Successful (See subroutine CRTHMX for error numbers.)

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle

- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **Tc** – [double&, out] :: Critical temperature [K]
- **Pc** – [double&, out] :: Critical pressure [kPa]
- **Dc** – [double&, out] :: Critical density [mol/L]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **CRTPNTcpp**(const *Handle* model_hdl, const std::vector<double> &z, double &Tc, double &Pc, double &Dc, int &ierr, std::string &herr, const int herr_length = 255)

Subroutine for the determination of the true critical point of a mixture with the use of the method of Michelsen (1984).

The routine requires good initial guess values of Pc and Tc.

On convergence, the values of bb and cc should be close to zero and dd > 0 for a two-phase critical point. bb=0, cc=0, and dd <= 0 for an unstable critical point.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **z** – [const std::vector<double>&, in] :: Composition [vector of mole fractions], maximum of 20 elements
- **Tc** – [double&, inout] :: Critical temperature [K]; set to initial guess value
- **Pc** – [double&, inout] :: Critical pressure [kPa]; set to initial guess value
- **Dc** – [double&, out] :: Critical density [mol/l]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **CSATKcpp**(const *Handle* model_hdl, const int icomp, const double T, const int kph, double &P, double &D, double &Csat, int &ierr, std::string &herr, const int herr_length = 255)

Compute the heat capacity along the saturation line as a function of temperature for a given component.

Csat can be calculated in several ways

$$\begin{aligned} \text{Csat} &= T \cdot (dS/dT[\text{sat}]) \\ \text{Csat} &= C_p - T \cdot (dV/dT)(dP/dT[\text{sat}]) \text{ with } dV/dT \text{ at constant pressure} \\ \text{Csat} &= C_p - \beta / D \cdot h_{\text{vap}} / (v_{\text{liq}} - v_{\text{vap}}) \end{aligned}$$

where beta is the volume expansivity

Flags flags

1 Liquid calculation **2** Vapor calculation

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **icomp** – [const int, in] :: Component number in mixture (1..nc); 1 for pure fluid
- **T** – [const double, in] :: Temperature [K]
- **kph** – [const int, in] :: Phase flag
- **P** – [double&, out] :: Saturated pressure [kPa]
- **D** – [double&, out] :: Saturated molar density [mol/L]
- **Csat** – [double&, out] :: Saturated heat capacity [J/mol-K]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void CSTARcpp(const Handle model_hdl, const double T, const double P, const double v, const  
std::vector<double> &z, double &Cs, double &Ts, double &Ds, double &Ps, double  
&ws, int &ierr, std::string &herr, const int herr_length = 255)
```

Calculate the critical flow factor, C^* , for nozzle flow of a gas (subroutine was originally named CCRIT).

Flags flags

0 Successful **151** CSTAR did not converge

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **P** – [const double, in] :: Pressure [kPa]
- **v** – [const double, in] :: Plenum velocity [m/s] (Should generally be set to 0 for calculating stagnation conditions.)
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **Cs** – [double&, out] :: Critical flow factor [dimensionless]
- **Ts** – [double&, out] :: Nozzle throat temperature [K]

- **Ds** – [double&, out] :: Nozzle throat molar density [mol/L]
- **Ps** – [double&, out] :: Nozzle throat pressure [kPa]
- **ws** – [double&, out] :: Nozzle throat speed of sound [m/s]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: Error string length (default: 255)
-

void **CV2PKcpp**(const *Handle* model_hdl, const int icomp, const double T, double &D, double &Cv2p, double &Csat, int &ierr, std::string &herr, const int herr_length = 255)

Compute the isochoric heat capacity in the two phase (liquid+vapor) region.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **icomp** – [const int, in] :: Component number in mixture (1..nc); 1 for pure fluid
- **T** – [const double, in] :: Temperature [K]
- **D** – [double&, in/out] :: Density [mol/L] if known. If D=0, then a saturated liquid state is assumed.
- **Cv2p** – [double&, out] :: Isochoric two-phase heat capacity [J/mol-K]
- **Csat** – [double&, out] :: Saturation heat capacity [J/mol-K] (Although there is already a Csat routine in Refprop, it is also returned here. However, the calculation speed is slower than Csat.)
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **CVCPcpp**(const *Handle* model_hdl, const double T, const double D, const std::vector<double> &z, double &Cv, double &Cp)

Calculate Cv and Cp for a mixture given temperature, density and composition.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **Cv** – [double&, out] :: Isochoric heat capacity [J/mol-K]
- **Cp** – [double&, out] :: Isobaric heat capacity [J/mol-K]

void **CVCPKcpp**(const *Handle* model_hdl, const int icom, const double T, const double D, double &Cv, double &Cp)

Calculate Cv and Cp for a pure fluid in a specified mixture given temperature and density

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **icom** – [const int, in] :: Component number in mixture (1..nc); 1 for pure fluid
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **Cv** – [double&, out] :: Isochoric heat capacity [J/mol-K]
- **Cp** – [double&, out] :: Isobaric heat capacity [J/mol-K]

void **DBDTcpp**(const *Handle* model_hdl, const double T, const std::vector<double> &z, double &dBt)

Compute the 1st derivative of B [dB/dT (L/mol-K)] as a function of temperature T (K) and composition x (vector of mole fractions). This routine approximates dB/dT. For pure fluids, the routine VIRBCD is exact.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **dBt** – [double, out] :: 1st derivative of B with respect to T [L/(mol-K)]

void **DDDPcpp**(const *Handle* model_hdl, const double T, const double D, const std::vector<double> &z, double &dDdP)

Compute partial derivative of density w.r.t. pressure at constant temperature as a function of temperature, density, and composition. See warning in subroutines THERM or ALLPROPS.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **dDdP** – [double&, out] :: dD/dP at constant temperature [mol/L-kPa]

void **DDDTcpp**(const *Handle* model_hdl, const double T, const double D, const std::vector<double> &z, double &dDdT)

Compute partial derivative of density w.r.t. temperature at constant pressure as a function of temperature, density, and composition. See warning in subroutines THERM or ALLPROPS.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **dDdT** – [double&, out] :: dD/dT at constant pressure [mol/L-K]

void **DEFL1cpp**(const *Handle* model_hdl, const double D, const double e, const std::vector<double> &z, double &T, int &ierr, std::string &herr, const int herr_length = 255)

Iterate for single-phase temperature as a function of density, energy, and composition.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **D** – [const double, in] :: Density [mol/L]
- **e** – [const double, in] :: Internal energy [J/mol]
- **z** – [const std::vector<double>&, in] :: Bulk composition (vector of mole fractions), maximum of 20 elements
- **T** – [double&, out] :: Temperature [K]
- **ierr** – [int&, out] :: Error code
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **DERVPVTCpp**(const *Handle* model_hdl, const double T, const double D, const std::vector<double> &z, double &dPdD, double &dPdT, double &d2PdD2, double &d2PdT2, double &d2PdTD, double &dDdP, double &dDdT, double &d2DdP2, double &d2DdT2, double &d2DdPT, double &dTdP, double &dTdD, double &d2TdP2, double &d2TdD2, double &d2TdPD)

Compute 1st and 2nd order derivatives of temperature, pressure, and density from core functions for Helmholtz energy equations only. See warning in subroutines THERM or ALLPROPS.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **dPdD** – [double&, out] :: dP/dD at constant T [kPa-L/mol]
- **dPdT** – [double&, out] :: dP/dT at constant D [kPa/K]

- **d2PdD2** – [double&, out] :: d^2P/dD^2 at constant T [kPa-L²/mol²]
- **d2PdT2** – [double&, out] :: d^2P/dT^2 at constant D [kPa/K²]
- **d2PdTD** – [double&, out] :: $d^2P/dTdD$ [kPa-L/K-mol]
- **dDdP** – [double&, out] :: dD/dP at constant T [mol/L-kPa]
- **dDdT** – [double&, out] :: dD/dT at constant P [mol/L-K]
- **d2DdP2** – [double&, out] :: d^2D/dP^2 at constant T [mol/L-kPa²]
- **d2DdT2** – [double&, out] :: d^2D/dT^2 at constant P [mol/L-K²]
- **d2DdPT** – [double&, out] :: $d^2D/dPdT$ [mol/L-kPa-K]
- **dTdP** – [double&, out] :: dT/dP at constant D [K/kPa]
- **dTdD** – [double&, out] :: dT/dD at constant P [K-L/mol]
- **d2TdP2** – [double&, out] :: d^2T/dP^2 at constant D [K/kPa²]
- **d2TdD2** – [double&, out] :: d^2T/dD^2 at constant P [K-L²/mol²]
- **d2TdPD** – [double&, out] :: $d^2T/dPdD$ [K-L/kPa-mol]

void **DHD1cpp**(const *Handle* model_hdl, const double T, const double D, const std::vector<double> &z, double &dhdt_d, double &dhdt_p, double &dhdd_t, double &dhdd_p, double &dhdp_t, double &dhdp_d)

Compute partial derivatives of enthalpy w.r.t. T, P, or D at constant T, P, or D as a function of temperature, density, and composition. See warning in subroutines THERM or ALLPROPS.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **dhdt_d** – [double&, out] :: dH/dT at constant density [J/mol-K]
- **dhdt_p** – [double&, out] :: dH/dT at pressure [J/mol-K]
- **dhdd_t** – [double&, out] :: dH/dD at constant temperature [(J/mol)/(mol/L)]
- **dhdd_p** – [double&, out] :: dH/dD at constant pressure [(J/mol)/(mol/L)]
- **dhdp_t** – [double&, out] :: dH/dP at constant temperature [J/mol-kPa]
- **dhdp_d** – [double&, out] :: dH/dP at constant density [J/mol-kPa]

void **DHFL1cpp**(const *Handle* model_hdl, const double D, const double h, const std::vector<double> &z, double &T, int &ierr, std::string &herr, const int herr_length = 255)

Iterate for single-phase temperature as a function of density, enthalpy, and composition.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **D** – [const double, in] :: Density [mol/L]
- **h** – [const double, in] :: Enthalpy [J/mol]
- **z** – [const std::vector<double>&, in] :: Bulk composition (vector of mole fractions), maximum of 20 elements
- **T** – [double&, out] :: Temperature [K]
- **ierr** – [int&, out] :: Error code
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **DIELECcpp**(const *Handle* model_hdl, const double T, const double D, const std::vector<double> &z, double &de)

Compute dielectric constant as a function of temperature, density, and composition.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **de** – [double&, out] :: Dielectric constant [-]

void **DLSATKcpp**(const *Handle* model_hdl, const int icomp, const double T, double &D, int &ierr, std::string &herr, const int herr_length = 255)

Compute pure fluid saturated liquid density with appropriate equation.

Flags flags

0 Successful **121** Temperature greater than critical point temperature **501** No equation available

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **icomp** – [const int, in] :: Component i
- **T** – [const double, in] :: Temperature [K]
- **D** – [double&, out] :: Saturated liquid density [mol/L]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

–

```
void DPDDcpp(const Handle model_hdl, const double T, const double D, const std::vector<double>
            &z, double &dPdD)
```

Compute partial derivative of pressure w.r.t. density at constant temperature as a function of temperature, density, and composition. See warning in subroutines THERM or ALLPROPS.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **dPdD** – [double&, out] :: dP/dD at constant temperature [kPa-L/mol]

```
void DPDD2cpp(const Handle model_hdl, const double T, const double D, const std::vector<double>
            &z, double &d2PdD2)
```

Compute second partial derivative of pressure w.r.t. density at constant temperature as a function of temperature, density, and composition. See warning in subroutines THERM or ALLPROPS.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **d2PdD2** – [double&, out] :: d^2P/dD^2 at constant temperature [kPa-L²/mol²]

```
void DPDTcpp(const Handle model_hdl, const double T, const double D, const std::vector<double>
            &z, double &dPdT)
```

Compute partial derivative of pressure w.r.t. temperature at constant density as a function of temperature, density, and composition. See warning in subroutines THERM or ALLPROPS.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **dPdT** – [double&, out] :: dP/dT at constant density [kPa/K]

```
void DPTSATKcpp(const Handle model_hdl, const int icomp, const double T, const int kph, double
                &P, double &D, double &Csat, double &dPdT, int &ierr, std::string &herr, const
                int herr_length = 255)
```

Compute the heat capacity and dP/dT along the saturation line as a function of temperature for a given component. See also subroutine CSATK.

Flags flags

1 Liquid calculation 2 Vapor calculation

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **icomp** – [const int, in] :: Component number in mixture (1..nc); 1 for pure fluid
- **T** – [const double, in] :: Temperature [K]
- **kph** – [const int, in] :: Phase flag
- **P** – [double&, out] :: Saturated pressure [kPa]
- **D** – [double&, out] :: Saturated molar density [mol/L]
- **Csat** – [double&, out] :: Saturated heat capacity [J/mol-K] (same as that called from CSATK)
- **dPdT** – [double&, out] :: dP/dT along the saturation line [kPa/K] (this is not dP/dT at the saturation line for the single phase state, but the change in saturated vapor pressure as the saturation temperature changes.)
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

–

```
void DSD1cpp(const Handle model_hdl, const double T, const double D, const std::vector<double>
             &z, double &dsdt_d, double &dsdt_p, double &dsdd_t, double &dsdd_p, double
             &dsdp_t, double &dsdp_d)
```

Compute partial derivatives of entropy w.r.t. T, P, or D at constant T, P, or D as a function of temperature, density, and composition. See warning in subroutines THERM or ALLPROPS.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements

- **dsdt_d** – [double&, out] :: dS/dT at constant density [J/mol-K²]
- **dsdt_p** – [double&, out] :: dS/dT at pressure [J/mol-K²]
- **dsdd_t** – [double&, out] :: dS/dD at constant temperature [(J/mol-K)/(mol/L)]
- **dsdd_p** – [double&, out] :: dS/dD at constant pressure [(J/mol-K)/(mol/L)]
- **dsdp_t** – [double&, out] :: dS/dP at constant temperature [J/mol-K-kPa]
- **dsdp_d** – [double&, out] :: dS/dP at constant density [J/mol-K-kPa]

void **DSFL1cpp**(const *Handle* model_hdl, const double D, const double s, const std::vector<double> &z, double &T, int &ierr, std::string &herr, const int herr_length = 255)

Iterate for single-phase temperature as a function of density, entropy, and composition.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **D** – [const double, in] :: Density [mol/L]
- **s** – [const double, in] :: Entropy [J/mol-K]
- **z** – [const std::vector<double>&, in] :: Bulk composition (vector of mole fractions), maximum of 20 elements
- **T** – [double&, out] :: Temperature [K]
- **ierr** – [int&, out] :: Error code
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **DVSATKcpp**(const *Handle* model_hdl, const int icomp, const double T, double &D, int &ierr, std::string &herr, const int herr_length = 255)

Compute pure fluid saturated vapor density with appropriate equation.

Flags flags

0 Successful **121** Temperature greater than critical point temperature **501** No equation available

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **icomp** – [const int, in] :: Component i
- **T** – [const double, in] :: Temperature [K]
- **D** – [double&, out] :: Saturated vapor density [mol/L]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string

- **herr_length** – [const int, in] :: length of variable herr (default: 255)

–

void **ENTHALcpp**(const *Handle* model_hdl, const double T, const double D, const std::vector<double> &z, double &h)

Calculate enthalpy for a mixture given temperature, density and composition

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **h** – [double&, out] :: Enthalpy [J/mol]

void **ENTROcpp**(const *Handle* model_hdl, const double T, const double D, const std::vector<double> &z, double &s)

Calculate entropy for a mixture given temperature, density and composition

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **s** – [double&, out] :: Entropy [J/mol-K]

void **ERRMSGcpp**(const *Handle* model_hdl, const int ierr, std::string &herr, const int herr_length = 255)

Retrieve the last error message saved in calls to ERRNUM (but only if the ierr variable is not equal to zero).

Write error messages to default output if iErrPrnt is active. The variable iErrPrnt in the common blocks must always be zero when compiling the DLL.

Outputs depend on variable iErrPrnt in the common blocks

- iErrPrnt= 0 - Error string not written (default)
- iErrPrnt=-1 - Error string written to screen
- iErrPrnt= 1 - Error string written to screen only if ierr is positive
- iErrPrnt=3,-3 - Same as 1 and -1, but program also pauses

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **ierr** – [const int, in] :: Error number from the last call to ERRNUM
- **herr** – [std::string&, out] :: Associated error string (character*255)
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void EXCESScpp(const Handle model_hdl, const double T, const double P, const std::vector<double>
    &z, const int kph, double &D, double &vE, double &eE, double &hE, double &sE,
    double &aE, double &gE, int &ierr, std::string &herr, const int herr_length = 255)
```

Compute excess properties as a function of temperature, pressure, and composition.

Flags flags

1 Liquid **2** Vapor **0** Stable phase

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **P** – [const double, in] :: Pressure [kPa]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **kph** – [const int, in] :: Phase flag
- **D** – [double&, in/out] :: Molar density [mol/L] (Send a negative density to the routine to use it as an initial guess.)
- **vE** – [double&, out] :: Excess volume [L/mol]
- **eE** – [double&, out] :: Excess energy [J/mol]
- **hE** – [double&, out] :: Excess enthalpy [J/mol]
- **sE** – [double&, out] :: Excess entropy [J/mol-K]
- **aE** – [double&, out] :: Excess Helmholtz energy [J/mol]
- **gE** – [double&, out] :: Excess Gibbs energy [J/mol]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: Error string length (default: 255)
- **_** *


```
void FGCTYcpp(const Handle model_hdl, const double T, const double D, const std::vector<double>
              &z, std::vector<double> &f)
```

Compute fugacity for each of the nc components of a mixture by analytical differentiation of the dimensionless residual Helmholtz energy. These are based on derivations in the GERG-2004 document for natural gas.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **f** – [std::vector<double>&, out] :: Fugacities [kPa]

```
void FGCTY2cpp(const Handle model_hdl, const double T, const double D, const
               std::vector<double> &z, std::vector<double> &f, int &ierr, std::string &herr, const
               int herr_length = 255)
```

Compute fugacity for each of the nc components of a mixture by analytical differentiation of the dimensionless residual Helmholtz energy. These are based on derivations in the GERG-2004 document for natural gas.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **f** – [std::vector<double>&, out] :: Fugacities [kPa]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: Error string length

```
void FLAGScpp(const Handle model_hdl, const std::string &hFlag, const int jFlag, int &kFlag, int
              &ierr, std::string &herr, const int hFlag_length = 255, const int herr_length = 255)
```

Set flags for desired behavior from the program.

Table 2: Table of flags in FLAGS function

hFlag	jFlag
Re- turn er- rors	• 0 - Return only final messages (default). • 1 - Return all intermediate messages. • 2 - Do not return messages. This flag is not reset with a new call to SETUP.
Write er- rors	• 0 - Error strings not written to the screen (default). • -1 - Error string written to the screen. • 1 - Error string written to the screen only if ierr is positive. • 3,-3 - Same as 1 and -1, but program also pauses. This flag is not reset with a new call to SETUP.
Dir search	• 0 - Search for fluid files in alternate directories (as defined in OPENFL) (default). • 1 - Do not search in directories other than the one set by the call to SETPATH, except for a ‘fluids’ subdirectory within the folder given in SETPATH. If the fluid files for the reference fluid(s) are not in the SETPATH directory, then transport properties may not be calculated. • 2 - Make no additional checks if the fluid file is not found after the first attempt to open the file (for example, checking upper and lower case). This flag is never reset.
Cp0P	• 1 - Change the ideal gas equation to Cp0. • 2 - Change the ideal gas equation to PH0. The default is set by the value in the fluid file. Calling SETUP resets the equation to its default state as given in the fluid file.
PX0	• 0 - Use the fluid file as is for the ideal gas equation (default). • 1 - Use the PX0 (or PH0 when no PX0 is available) for all calculations and turn off the effect of SETREF. For mixtures, the reference state of “each pure component” will be used. Reference states cannot be changed with this option on, the state is preset by values in the fluid file. This flag is never reset. When setting up the fluids through a call to the REFPROP subroutine, the SETREF flag described below will have no effect if the PX0 option has already been turned on. Warning: Don’t use the CP0PH0 flag to switch back to Cp0, recall this routine with jFlag=0.
Skip SE- TREF	• 0 - Call the SETREF routine during the call to SETUP to set the reference state (default). The only reason to call this would be to increase the speed of setting up new fluids because the SETREF routine has to make complex saturation calculations. • 1 - Skip the call to SETREF. However, this means energy, enthalpy, and entropy will not be correct (but only by an offset to their usual values). This must be called before the call to SETUP, and is never reset.
Mix- ture ref- er- ence or SE-	• 0 - Do nothing (default). • 1 - When calling subroutine REFPROP, call SETREF first with a value of 1 for the second entry. • 2 - When calling subroutine REFPROP, call SETREF first with a value of 2 for the second entry. (See subroutine SETREF for details.) This must be called before the call to REFPROP (the subroutine in

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **hFlag** – [const std::string&, in] :: Indicator for the option to set (letters in the string are case insensitive).
- **jFlag** – [const int, in] :: Flag to choose what to do in each option. Send -999 to just obtain the current value of the flag.
- **kFlag** – [int&, out] :: Current setting of the flag for the option identified by hFlag. (Returned regardless of the value of jFlag.)
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **hFlag_length** – [const int, in] :: length of variable hFlag (default: 255)
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **FPVcpp**(const *Handle* model_hdl, const double T, const double D, const double P, const std::vector<double> &z, double &Fpvx)

Compute the supercompressibility factor, Fpv.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **P** – [const double, in] :: Pressure [kPa]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **Fpvx** – [double&, out] :: $Fpv = \text{SQRT}[Z(60 \text{ F}, 14.73 \text{ psia})/Z(T,P)]$

void **FUGCOFcpp**(const *Handle* model_hdl, const double T, const double D, const std::vector<double> &z, std::vector<double> &phi, int &ierr, std::string &herr, const int herr_length = 255)

Compute the fugacity coefficient for each of the nc components of a mixture.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **phi** – [std::vector<double>&, out] :: Fugacity coefficients [-]

- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: Error string length

```
void GETENUMcpp(const Handle model_hdl, const int iFlag, const std::string &hEnum, int &iEnum,  
               int &ierr, std::string &herr, const int hEnum_length = 255, const int herr_length =  
               255)
```

Translate a string of letters into an integer value that can be used in calls to ALLPROPS0 to increase the speed of property calculations by eliminating string comparisons (which are time expensive).

This can be done once at the beginning of a program for all properties that will be used, and stored for later use as needed.

The input strings possible are described in subroutines ALLPROPS and GETUNIT.

Flags flags

0 Check all strings possible **1** Check strings for property units only (e.g., SI, English, etc.)
2 Check property strings and those in #3 only. **3** Check property strings only that are not functions of T and D (for example, the critical point, acentric factor, limits of the EOS, etc.)

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **iFlag** – [const int, in] :: Flag to specify which type of enumerated value to return
- **hEnum** – [const std::string&, in] :: The string that will be used to return the enumerated value. Only uppercase letters are allowed to decrease the time required to process the values.
- **iEnum** – [int&, out] :: The enumerated value that matches the string sent in hEnum.
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **hEnum_length** – [const int, in] :: length of variable hEnum (default: 255)
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void GETFIJcpp(const Handle model_hdl, const std::string &hmodij, std::vector<double> &fij,  
              std::string &hfij, std::string &hmxrul, const int hmodij_length = 3, const int  
              hfij_length = 255, const int hmxrul_length = 255)
```

Retrieve parameter info for a specified mixing rule.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle

- **hmodij** – [const std::string&, in] :: Mixing rule for the binary pair i,j (e.g., LJ6 or KW0) (character*3)
- **fij** – [std::vector<double>&, out] :: Binary mixture parameters (array of dimension nmixpar; currently nmixpar is set to 6). The parameters will vary depending on hmodij.
- **hfij** – [std::string&, out] :: Description of the binary mixture parameters (character*8 array of dimension nmixpar)
- **hmxrul** – [std::string&, out] :: Description of the mixing rule (character*255)
- **hmodij_length** – [const int, in] :: length of variable hmodij (default: 3)
- **hfij_length** – [const int, in] :: length of variable hfij (default: 255)
- **hmxrul_length** – [const int, in] :: length of variable hmxrul (default: 255)

```
void GETKTVcpp(const Handle model_hdl, const int icomp, const int jcomp, std::string &hmodij,
               std::vector<double> &fij, std::string &hFmix, std::string &hfij, std::string &hbinp,
               std::string &hmxrul, const int hmodij_length = 3, const int hFmix_length = 255,
               const int hfij_length = 255, const int hbinp_length = 255, const int hmxrul_length =
               255)
```

Retrieve mixture model and parameters for a specified binary mixture.

This subroutine should not be called until after SETUP has been called. The order of icomp and jcomp do not matter, the routine returns the parameters as stored in the HMX.BNC file. To determine if the compositions are backwards, call HMXORDERcpp. If calling SETMIXTUREcpp with the same parameters, an error will be returned if the components are backwards.

Kunz-Wagner model (KW0)	Lemmon-Jacobsen model (LJ6)
fij(1) = betaT	fij(1) = zeta
fij(2) = gammaT	fij(2) = xi
fij(3) = betaV	fij(3) = Fij
fij(4) = gammaV	fij(4) = beta
fij(5) = Fij	fij(5) = gamma
fij(6) = 'not used'	fij(6) = 'not used'

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **icomp** – [const int, in] :: Component i
- **jcomp** – [const int, in] :: Component j
- **hmodij** – [std::string&, out] :: Mixing rule for the binary pair i,j (e.g., KW0, LJ6, XR0, or LIN) (character*3)

- **fij** – [std::vector<double>&, out] :: Binary mixture parameters (array of dimension nmixpar; currently nmixpar is set to 6); the parameters will vary depending on hmodij
- **hFmix** – [std::string&, out] :: File name (character*255) containing parameters for the binary mixture model
- **hfij** – [std::string&, out] :: Description of the binary mixture parameters (character*8 array of dimension nmixpar) The parameters will vary depending on hmodij.
- **hbinp** – [std::string&, out] :: Documentation for the binary parameters (character*255)
- **hmxrul** – [std::string&, out] :: Description of the mixing rule (character*255)
- **hmodij_length** – [const int, in] :: length of variable hmodij (default: 3)
- **hFmix_length** – [const int, in] :: length of variable hFmix (default: 255)
- **hfij_length** – [const int, in] :: length of variable hfij (default: 255)
- **hbinp_length** – [const int, in] :: length of variable hbinp (default: 255)
- **hmxrul_length** – [const int, in] :: length of variable hmxrul (default: 255)

void **GETMODcpp**(const *Handle* model_hdl, const int icomp, const std::string &htype, std::string &hcode, std::string &hcite, const int htype_length = 3, const int hcode_length = 3, const int hcite_length = 255)

Retrieve citation information for the property models used.

Flags flags

EOS Equation of state
CP0 Ideal-gas heat capacity
ETA Viscosity
TCX Thermal conductivity
TKK Thermal conductivity critical enhancement
STN Surface tension
***DE *** Dielectric constant
MLT Melting line (i.e., freezing line)
SBL Sublimation line
***PS *** Vapor pressure equation
***DL *** Saturated liquid density equation
***DV *** Saturated vapor density equation

Flags flags

FEQ Helmholtz energy model
ECS Extended corresponding states (all fluids)
VS1 The ‘composite’ model for R134a, R152a, NH3, etc.
VS2 Younglove-Ely model for hydrocarbons
VS4 Generalized friction theory of Quinones-Cisneros and Dieters
VS5 Chung et al. model
VS6 Vesovic form of VS1 model
VS7 Polynomial/exponential model
TC1 The ‘composite’ model for R134a, R152a, etc.
TC2 Younglove-Ely model for hydrocarbons
TC5 Predictive model of Chung et al. (1988)
ST1 surface tension as $f(\tau)$; $\tau = 1 - T/T_c$

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **icomp** – [const int, in] :: Pointer specifying component number; zero and negative values are used for ECS reference fluid(s)
- **htype** – [const std::string&, in] :: Flag indicating which model is to be retrieved (character*3)
- **hcode** – [std::string&, out] :: Component model used for property specified in htype (character*3)
- **hcite** – [std::string&, out] :: Component model used for property specified in htype; the first 3 characters repeat the model designation of hcode and the remaining are the citation for the source (character*255)
- **htype_length** – [const int, in] :: length of variable htype (default: 3)
- **hcode_length** – [const int, in] :: length of variable hcode (default: 3)
- **hcite_length** – [const int, in] :: length of variable hcite (default: 255)

void **GETREFDIRcpp**(const *Handle* model_hdl, std::string &hpth, const int hpth_length = 255)

Get the path where the original fluid files are located. See SETREFDIRcpp for more information.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **hpth** – [std::string&, out] :: Location of the original fluid files (character*255)
- **hpth_length** – [const int, in] :: length of variable hpth (default: 255)

```
void GERG04cpp(const Handle model_hdl, const int ncomp, const int iFlag, int &ierr, std::string
               &herr, const int herr_length = 255)
```

Use the GERG 2008 formulation for all pure fluid and mixture calculations.

This is a duplicate of the GERG08 routine and is meant only for use with older versions of Refprop. This subroutine must be called before SETUP; it need not be called at all if the default (NIST-recommended) models are desired. To turn off the GERG settings, call this routine again with iFlag=0, and then call the SETUP routine to reset the parameters of the equations of state. Once this routine is called, it need not be called again to keep the GERG model active, even when calling SETUP.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **ncomp** – [const int, in] :: Number of components (1 for pure fluid)
- **iFlag** – [const int, in] :: Set to 1 to load the GERG 2008 equations, set to 0 for defaults
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string (returned from SETMOD)
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void GERG08cpp(const Handle model_hdl, const int ncomp, const int iFlag, int &ierr, std::string
               &herr, const int herr_length = 255)
```

Use the GERG 2008 formulation for all pure fluid and mixture calculations.

This subroutine must be called before SETUP; it need not be called at all if the default (NIST-recommended) models are desired. To turn off the GERG settings, call this routine again with iFlag=0, and then call the SETUP routine to reset the parameters of the equations of state. Once this routine is called, it need not be called again to keep the GERG model active, even when calling SETUP.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **ncomp** – [const int, in] :: Number of components (1 for pure fluid)
- **iFlag** – [const int, in] :: Set to 1 to load the GERG 2008 equations, set to 0 for defaults
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string (returned from SETMOD)
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void GIBBScpp(const Handle model_hdl, const double T, const double D, const std::vector<double>
              &z, double &ar, double &gr)
```

Compute residual Helmholtz and Gibbs energies as functions of temperature, density, and composition from core functions, calculated as:

$$G(T,D) - G_0(T,P^*) = G(T,D) - G_0(T,D) + RT\ln(RTD/P^*)$$

where G_0 is the ideal-gas state and P^* is a reference pressure that is equal to the current pressure of interest. Since G_r is used only as a difference in phase equilibria calculations where the temperature and pressure of the phases are equal, the (RT/P^*) part of the log term will cancel and is omitted. Normal (not residual) A and G are computed by subroutine `AG`.

See warning in subroutines `THERM` or `ALLPROPS`.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **ar** – [double&, out] :: Residual Helmholtz energy [J/mol]
- **gr** – [double&, out] :: Residual Gibbs free energy [J/mol]

```
void HEATcpp(const Handle model_hdl, const double T, const double D, const std::vector<double>
            &z, double &hg, double &hn, int &ierr, std::string &herr, const int herr_length = 255)
```

Compute the ideal-gas gross and net heating values.

Flags flags

0 Successful **662** Not all heating values available **665** Error in chemical formula

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **hg** – [double&, out] :: Gross (or superior) heating value [J/mol]
- **hn** – [double&, out] :: Net (or inferior) heating value [J/mol]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void HEATFRMcpp(const Handle model_hdl, const double T, const double D, const
               std::vector<double> &z, double &hFrm, int &ierr, std::string &herr, const int
               herr_length = 255)
```

Compute the heat of formation.

The heat of formation is the heat required to form a compound from its constituent elements, with the standard state defined as 298.15 K for the ideal gas.

Flags flags

0 Successful **662** Not all heating values available **664** Unknown species in chemical formula
665 Error in chemical formula

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L] (not used)
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **hFrm** – [double&, out] :: Heat of formation [J/mol]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void HMXORDERcpp(const Handle model_hdl, const int i, const int j, std::string &hcasi, std::string  
                  &hcasj, int &iFlag, int &ierr, std::string &herr, const int hIDi_length = 255,  
                  const int hIDj_length = 255, const int herr_length = 255)
```

Return the ID numbers in the order given in the HMX.BNC file, and a flag that indicates if the loaded fluids are in the same order.

Flags flags

0 Pair is backwards **1** Pair is in correct order (or if i=j) **2** Pair is not in HMX.BNC

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **i** – [const int, in] :: Component i
- **j** – [const int, in] :: Component j
- **hcasi** – [std::string&, out] :: ID number of the first fluid stored in the HMX.BNC file for the i,j pair (character*255)
- **hcasj** – [std::string&, out] :: ID number of the second fluid stored in the HMX.BNC file for the i,j pair (character*255)

- **iFlag** – [int&, out] :: Flag to indicate if loaded fluids are in the same order as the i,j pair
- **ierr** – [int&, out] :: Error number
- **herr** – [std::string&, out] :: Error message
- **hIDi_length** – [const int, in] :: length of variable hcas_i (default: 255)
- **hIDj_length** – [const int, in] :: length of variable hcas_j (default: 255)
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **IDCRVcpp**(const *Handle* model_hdl, const double D, const std::vector<double> &z, double &T, int &ierr, std::string &herr, const int herr_length = 255)

Calculate the temperature at the input density where the compressibility factor crosses from less than 1 to greater than 1 (i.e., $Z=1$).

This line starts at zero density at the temperature where $B=0$, and passes into the liquid phase without crossing the two-phase. The argument z in this routine is an array with the mole fractions of the mixture. If the input T is non-zero, it is used as the initial guess.

Flags flags

0 Successful **151** Iteration failed to converge

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **D** – [const double, in] :: Density [mol/l]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **T** – [double&, in/out] :: Temperature [K]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **INFOcpp**(const *Handle* model_hdl, const int icomp, double &wmm, double &Ttrp, double &Tnbpt, double &Tc, double &Pc, double &Dc, double &Zc, double &acf, double &dip, double &Rgas)

Provides fluid constants for the specified component.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **icomp** – [const int, in] :: Component number in mixture; 1 for pure fluid
- **wmm** – [double&, out] :: Molar mass (molecular weight) [g/mol]

- **Ttrp** – [double&, out] :: Triple point temperature [K]
- **Tnbpt** – [double&, out] :: Normal boiling point temperature [K]
- **Tc** – [double&, out] :: Critical temperature [K]
- **Pc** – [double&, out] :: Critical pressure [kPa]
- **Dc** – [double&, out] :: Critical density [mol/L]
- **Zc** – [double&, out] :: Compressibility factor at critical point [$P_c/(R_{gas} \cdot T_c \cdot D_c)$]
- **acf** – [double&, out] :: Acentric factor [-]
- **dip** – [double&, out] :: Dipole moment [debye]
- **Rgas** – [double&, out] :: Gas constant [J/mol-K]

void **JICRVcpp**(const *Handle* model_hdl, const double D, const std::vector<double> &z, double &T, int &ierr, std::string &herr, const int herr_length = 255)

Calculate the temperature along the Joule-Inversion curve for the input density.

This line starts at zero density at the temperature where B is at a maximum, and passes into the liquid phase without crossing the two-phase. It ends at very high pressures. The argument z in this routine is an array with the mole fractions of the mixture. If the input T is non-zero, it is used as the initial guess.

JI is equal to

$$\begin{aligned} & d(Z)/d(T) \text{ at constant } D \\ & \text{del} \cdot d^2(\text{alphan})/d(\text{del})/d(T) \\ & - \text{del} \cdot \tau \cdot d^2(\text{alphan})/d(\text{del})/d(\tau)/T \text{ (can ignore the } /T \text{ for finding } JI=0) \end{aligned}$$

$d(JI)/dT$:

$$\tau \cdot d^2(\text{alphan})/d(\text{del})/d(\tau) \cdot \text{del}^2/T^2$$

(One of the /T must be removed to match the one removed in the function.)

Flags flags

0 Successful **151** Iteration failed to converge

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **D** – [const double, in] :: Density [mol/l]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **T** – [double&, in/out] :: Temperature [K]
- **ierr** – [int&, out] :: Error flag

- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **JTCRVcpp**(const *Handle* model_hdl, const double D, const std::vector<double> &z, double &T, int &ierr, std::string &herr, const int herr_length = 255)

Calculate the temperature along the Joule-Thomson curve for the input density.

This line starts at zero density at the temperature where the Joule-Thomson property (dH/dT) is zero, and passes into the liquid phase without crossing the two-phase. It ends at a saturated liquid state far from the critical point. The argument z in this routine is an array with the mole fractions of the mixture. If the input T is non-zero, it is used as the initial guess.

Only the top part in the calculation of hjt is required, the other parts do not go to zero and thus do not contribute to finding $JT=0$.

Flags flags

0 Successful **151** Iteration failed to converge

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **D** – [const double, in] :: Density [mol/l]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **T** – [double&, in/out] :: Temperature [K]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **LIMITKcpp**(const *Handle* model_hdl, const std::string &htyp, const int icomp, const double T, const double D, const double P, double &Tmin, double &Tmax, double &Dmax, double &Pmax, int &ierr, std::string &herr, const int htyp_length = 3, const int herr_length = 255)

Calculate values of property limits for a specified model. This function is deprecated. Use LIM-ITXcpp instead.

This function is deprecated. Use subroutine LIMITX instead.

Flags flags

- ***EOS*** Equation of state
- ***ETA*** Viscosity
- ***TCX*** Thermal conductivity

STN Surface tension

Flags flags

0 All inputs within limits **-1** $1.5 \cdot T_{\max} > T > T_{\max}$ **1** $T < T_{\min}$ or $T > 1.5 \cdot T_{\max}$ **2** $D > D_{\max}$ or $D < 0$ **-4** $2 \cdot P_{\max} > P > P_{\max}$ **4** $P < 0$ or $P > 2 \cdot P_{\max}$ **8** Component composition < 0 or > 1 and/or composition sum $\neq 1$ **16** $P > P_{\text{melt}}$ **-16** $T < T_{\text{trp}}$ (important for water)

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **htyp** – [const std::string&, in] :: Flag indicating the model to check (character*3)
- **icomp** – [const int, in] :: Component number in mixture (1..nc)
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **P** – [const double, in] :: Pressure [kPa]
- **Tmin** – [double&, out] :: Minimum temperature for model specified by htyp [K]
- **Tmax** – [double&, out] :: Maximum temperature [K]
- **Dmax** – [double&, out] :: Maximum density [mol/L]
- **Pmax** – [double&, out] :: Maximum pressure [kPa]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **htyp_length** – [const int, in] :: length of variable htyp (default: 3)
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void LIMITScpp(const Handle model_hdl, const std::string &htyp, const std::vector<double> &z,  
               double &Tmin, double &Tmax, double &Dmax, double &Pmax, const int  
               htyp_length = 3)
```

Returns limits of a property model as a function of composition.

Pure fluid limits were read in from the *.fld files; for mixtures, a simple mole fraction weighting in reduced variables is used.

Flags flags

EOS Equation of state for thermodynamic properties

ETA Viscosity

TCX Thermal conductivity

STN Surface tension

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **htyp** – [const std::string&, in] :: Flag indicating which models are to be checked (character*3)
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **Tmin** – [double&, out] :: Minimum temperature for model specified by htyp [K]
- **Tmax** – [double&, out] :: Maximum temperature [K]
- **Dmax** – [double&, out] :: Maximum density [mol/L]
- **Pmax** – [double&, out] :: Maximum pressure [kPa]
- **htyp_length** – [const int, in] :: length of variable htyp (default: 3)

```
void LIMITXcpp(const Handle model_hdl, const std::string &htyp, const double T, const double D,
               const double P, const std::vector<double> &z, double &Tmin, double &Tmax,
               double &Dmax, double &Pmax, int &ierr, std::string &herr, const int htyp_length
               = 3, const int herr_length = 255)
```

Returns limits of a property model as a function of composition and/or checks inputs T, D, and P against those limits.

Pure fluid limits are read in from the *.fld files; for mixtures, a simple mole fraction weighting of the reduced variables is used.

Attempting calculations below the minimum temperature and/or above the maximum density may result in an error. These will often correspond to a physically unreasonable state; also many equations of state do not extrapolate reliably to lower T's and higher D's.

A warning is issued if the temperature is above the maximum but below 1.5 times the maximum. Pressures up to twice the maximum result in only a warning. Most equations of state may be extrapolated to higher T's and P's. Temperatures and/or pressures outside these extended limits will result in an error.

When calling with an unknown temperature, set T to -1 to avoid performing the melting line check. If inputs are not available, use T=300, P=0, and D=0.

If multiple inputs are outside limits, ierr=SUM(ABS(ierr)), with a positive sign if any error greater than zero (calculations not possible), or a negative sign for warnings only.

Flags flags

- ***EOS*** Equation of state
- ***ETA*** Viscosity
- ***TCX*** Thermal conductivity
- ***STN*** Surface tension

Flags flags

0 All inputs within limits **-1** $1.5 \cdot T_{\max} > T > T_{\max}$ **1** $T < T_{\min}$ or $T > 1.5 \cdot T_{\max}$ **2** $D >$

Dmax or D < 0 **-4** 2*Pmax > P > Pmax **4** P < 0 or P > 2*Pmax **8** Component composition < 0 or > 1 and/or composition sum <> 1 **16** P>Pmelt **-16** T<Ttrp (important for water)

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **htyp** – [const std::string&, in] :: Flag indicating the model to check (character*3)
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **P** – [const double, in] :: Pressure [kPa]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **Tmin** – [double&, out] :: Minimum temperature for model specified by htyp [K]
- **Tmax** – [double&, out] :: Maximum temperature [K]
- **Dmax** – [double&, out] :: Maximum density [mol/L]
- **Pmax** – [double&, out] :: Maximum pressure [kPa]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **htyp_length** – [const int, in] :: length of variable htyp (default: 3)
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **LIQSPNDLcpp**(const *Handle* model_hdl, const double T, const std::vector<double> &z, double &D, int &ierr, std::string &herr, const int herr_length = 255)

Find the liquid spinodal density for a given temperature.

If no spinodal exists, return the point of zero curvature. This only happens with a few of the older equations, these being argon, ethane, nitrogen, R22, and R124.

Flags flags

0 Successful **121** T>Tc **633** Failed to converge **-638** Spinodal not found, point of zero curvature returned

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **D** – [double&, out] :: Density at liquid spinodal [mol/L]

- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void MASSFLUXcpp(const Handle model_hdl, const double Tm, const double P, const
    std::vector<double> &z, const double beta, const double rf, double &fluxm,
    double &Cs, double &T0, double &P0, double &xMach, double &u, double
    &Ts, double &Ps, int &ierr, std::string &herr, const int herr_length = 255)
```

Calculate the theoretical mass flux for a CFV (critical flow venturi) of a gas. This is required for high beta; CSTARcpp can be used for low beta.

Flags flags

0 Successful **151** Iteration failed to converge

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **Tm** – [const double, in] :: Measured temperature [K]
- **P** – [const double, in] :: Upstream (static) pressure [kPa]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **beta** – [const double, in] :: Ratio of throat diameter to pipe diameter [-]
- **rf** – [const double, in] :: Recovery factor $[(T_m - T)/(T_0 - T)]$, (T is static temperature, T0 is the stagnation temperature)
- **fluxm** – [double&, out] :: Theoretical mass flux $[\text{kg}/(\text{m}^2\text{-s})]$
- **Cs** – [double&, out] :: Critical flow factor [-]
- **T0** – [double&, out] :: Stagnation temperature [K]
- **P0** – [double&, out] :: Stagnation pressure [kPa]
- **xMach** – [double&, out] :: Mach number (u/speed of sound) [-]
- **u** – [double&, out] :: Average axial velocity in approach pipe upstream of the CFV [m/s]
- **Ts** – [double&, out] :: Temperature at throat [K]
- **Ps** – [double&, out] :: Pressure at throat [kPa]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: Error string length (default: 255)

```
void MAXPcpp(const Handle model_hdl, const std::vector<double> &z, double &Tm, double &Pm,  
             double &Dm, int &ierr, std::string &herr, const int herr_length = 255)
```

Calculate values at the maximum pressure along the saturation line.

These are returned from the call to SATSPLN and apply only to the composition in the z() array sent to SATSPLN. For a pure fluid, the critical point is returned.

Flags flags

0 Successful **331** Splines not available for calculation **-362** Maximum pressure not known

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **Tm** – [double&, out] :: Temperature [K]
- **Pm** – [double&, out] :: Pressure [kPa]
- **Dm** – [double&, out] :: Density [mol/L]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void MAXTcpp(const Handle model_hdl, const std::vector<double> &z, double &Tm, double &Pm,  
             double &Dm, int &ierr, std::string &herr, const int herr_length = 255)
```

Calculate values at the maximum temperature along the saturation line.

These are returned from the call to SATSPLN and apply only to the composition in the z() array sent to SATSPLN. For a pure fluid, the critical point is returned.

Flags flags

0 Successful **331** Splines not available for calculation **-361** Maximum temperature not known

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **Tm** – [double&, out] :: Temperature [K]
- **Pm** – [double&, out] :: Pressure [kPa]
- **Dm** – [double&, out] :: Density [mol/L]

- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **MELTKcpp**(const *Handle* model_hdl, const int icomp, const double T, double &P, int &ierr, std::string &herr, const int herr_length = 255)

Compute melting line pressure with appropriate core model.

There are two functional forms for the melting line, labeled in the fluid files as ML1 and ML2: ML1: $P = P_{\text{red}} * P_r$ ML2: $P = P_{\text{red}} * \text{Exp}(P_r)$ where: $P_r = \text{Sum}[N_k * T_r^{t_k}] + \text{Sum}[N_k * (T_r - 1)^{t_k}] + \text{Sum}[N_k * (\text{Log } T_r)^{t_k}]$ $T_r = T / T_{\text{red}}$ In the fluid file, T_{red} and P_{red} (the reducing values) are given first, followed by the number of terms in each of the summations, and then followed by the coefficients N_k and exponents t_k (one term with N_k and t_k listed per line).

Flags flags

0 Successful **1** $T < T_{\text{trp}}$ **-4** $P < P_{\text{trp}}$ (for MELTP routine) **501** No equation available **502** Unknown melting line equation

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **icomp** – [const int, in] :: Component i (for water and heavy water, send -icomp to obtain the root with the lower pressure at $T < T_{\text{trp}}$)
- **T** – [const double, in] :: Temperature [K]
- **P** – [double&, out] :: Melting line pressure [kPa]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **MELTPcpp**(const *Handle* model_hdl, const double P, const std::vector<double> &z, double &T, int &ierr, std::string &herr, const int herr_length = 255)

Compute the melting line temperature as a function of pressure and composition.

Flags flags

0 Successful **-4** Pressure below triple point pressure **501** No equation available

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **P** – [const double, in] :: Melting line pressure [kPa]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements

- **T** – [double&, out] :: Temperature [K]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **MELTcpp**(const *Handle* model_hdl, const double T, const std::vector<double> &z, double &P, int &ierr, std::string &herr, const int herr_length = 255)

Compute the melting line pressure as a function of temperature and composition.

Flags flags

0 Successful **501** No equation available

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **P** – [double&, out] :: Melting line pressure [kPa]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **MLTH20cpp**(const *Handle* model_hdl, double &T, double &P1, double &P2)

Compute melting line of water, see fluid file for reference.

If flags of -998 or -999 are sent for the temperature, the value of the lowest temperature possible (251.165 K) is sent back in T, the pressure at that point is sent back in P1, and the density at that point is sent back in P2 if the flag -998 is used.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [double&, in/out] :: Temperature [K]; special flags -998 or -999 return Tmin in T
- **P1** – [double&, out] :: Higher melting line pressure [kPa]; above 273.16 K only P1 returns a physical answer
- **P2** – [double&, out] :: Lower melting line pressure [kPa]; between 251.165 and 273.16 K, two pressures are returned

```
void NAMEcpp(const Handle model_hdl, const int icomp, std::string &hnam, std::string &hn80,
             std::string &hcasn, const int hnam_length = 12, const int hn80_length = 80, const int
             hcasn_length = 12)
```

Provides name information for the specified component.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **icomp** – [const int, in] :: Component number in mixture; 1 for pure fluid
- **hnam** – [std::string&, out] :: Component name (character*12) (send icomp+1000 to get the fluid hash)
- **hn80** – [std::string&, out] :: Component name - long form (character*80). To return the file name used when SETUP was called (without path), send -icomp. If path is also needed, use PASSCMN. For example: call PASSCMN ('hdir',0,1,0,hfl,i,xx,arr,ierr,herr)
- **hcasn** – [std::string&, out] :: ID (Chemical Abstracts Service) number (character*12)
- **hnam_length** – [const int, in] :: length of variable hnam (default: 12)
- **hn80_length** – [const int, in] :: length of variable hn80 (default: 80)
- **hcasn_length** – [const int, in] :: length of variable hcasn (default: 12)

```
void PASSCMNcpp(const Handle model_hdl, const std::string &hvr, const int iset, const int icomp,
               const int jcomp, std::string &hstr, int &ilng, double &dbl, std::vector<double>
               &arr, int &ierr, std::string &herr, const int hvr_length = 255, const int hstr_length
               = 255, const int herr_length = 255)
```

Get or set values of variables in the common blocks.

Examples:

```
PASSCMNd11 ('txeos', 0, 3, 0, h, i, tmx, z, ierr, herr) // get Tmax of
↪ component 3
PASSCMNd11 ('dxeos', 1, 2, 0, h, i, dmx, z, ierr, herr) // set Dmax of
↪ component 2
PASSCMNd11 ('tz', 0, 1, 0, h, i, Tc, z, ierr, herr) // get reducing
↪ temperature of component 1
PASSCMNd11 ('ntermf', 0, 1, 0, h, nt, v, z, ierr, herr) // get number of
↪ terms in the Helmholtz equation for component 1
PASSCMNd11 ('coefhmx', 1, 1, 0, h, i, v, cf, ierr, herr) // set the
↪ coefficients in the Helmholtz equation for component 1
PASSCMNd11 ('acp0', 1, 5, 0, h, i, v, cp0, ierr, herr) // set the
↪ coefficients in the cp0 equation for component 5
PASSCMNd11 ('fprkij', 1, 1, 2, h, i, v, fpr, ierr, herr) // set the PR
↪ coefficient for the 1,2 binary
```

Flags flags

0 Get variable value **1** Set variable value

Flags flags

0 Successful **113** Inputs out of bounds **601** Variable name not recognized

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **hvr** – [const std::string&, in] :: Character string with the common variable's name
- **iset** – [const int, in] :: Flag to indicate the get/set condition
- **icomp** – [const int, in] :: Component number
- **jcomp** – [const int, in] :: Second component number for binary mixture variables
- **hstr** – [std::string&, out] :: Input or output for a character string
- **ilng** – [int&, out] :: Input or output for a long variable
- **dbl** – [double&, out] :: Input or output for a double precision variable
- **arr** – [std::vector<double>&, out] :: Input or output for a double precision array, maximum of 100 elements
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **hvr_length** – [const int, in] :: length of variable hvr (default: 255)
- **hstr_length** – [const int, in] :: length of variable hstr (default: 255)
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void PDFL1cpp(const Handle model_hdl, const double P, const double D, const std::vector<double>
    &z, double &T, int &ierr, std::string &herr, const int herr_length = 255)
```

Iterate for single-phase temperature as a function of pressure, density, and composition.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **P** – [const double, in] :: Pressure [kPa]
- **D** – [const double, in] :: Density [mol/L]
- **z** – [const std::vector<double>&, in] :: Bulk composition (vector of mole fractions), maximum of 20 elements
- **T** – [double&, out] :: Temperature [K]
- **ierr** – [int&, out] :: Error code
- **herr** – [std::string&, out] :: Error string

- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void PEFL1cpp(const Handle model_hdl, const double P, const double e, const std::vector<double>
    &z, const int kph, double &T, double &D, int &ierr, std::string &herr, const int
    herr_length = 255)
```

Iterate for single-phase temperature and density as a function of pressure, energy, and composition.

Flags flags

1 Liquid phase **2** Vapor phase

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **P** – [const double, in] :: Pressure [kPa]
- **e** – [const double, in] :: Internal energy [J/mol]
- **z** – [const std::vector<double>&, in] :: Bulk composition (vector of mole fractions), maximum of 20 elements
- **kph** – [const int, in] :: Phase flag
- **T** – [double&, out] :: Temperature [K]
- **D** – [double&, out] :: Density [mol/L]
- **ierr** – [int&, out] :: Error code
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void PHFL1cpp(const Handle model_hdl, const double P, const double h, const std::vector<double>
    &z, const int kph, double &T, double &D, int &ierr, std::string &herr, const int
    herr_length = 255)
```

Iterate for single-phase temperature and density as a function of pressure, enthalpy, and composition.

Flags flags

1 Liquid phase **2** Vapor phase

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **P** – [const double, in] :: Pressure [kPa]
- **h** – [const double, in] :: Enthalpy [J/mol]
- **z** – [const std::vector<double>&, in] :: Bulk composition (vector of mole fractions), maximum of 20 elements

- **kph** – [const int, in] :: Phase flag
- **T** – [double&, out] :: Temperature [K]
- **D** – [double&, out] :: Density [mol/L]
- **ierr** – [int&, out] :: Error code
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **PHI0cpp**(const *Handle* model_hdl, const int itau, const int idel, const double T, const double D, const std::vector<double> &z, double &phi00)

Compute the ideal-gas part of the reduced Helmholtz energy or its derivatives as functions of temperature and density for a mixture. While the real-gas part of the Helmholtz energy is calculated in terms of dimensionless temperature and density, the ideal-gas part is calculated in terms of absolute temperature and density. (This distinction is necessary for mixtures.) The Helmholtz energy consists of ideal-gas and residual (real-gas) terms; this routine calculates only the ideal part.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **itau** – [const int, in] :: Flag specifying the order of the temperature derivative
- **idel** – [const int, in] :: Flag specifying the order of the density derivative. (The density derivatives are not used in the calculation of any property.)
 - when itau = 0 and idel = 0, compute A0/RT
 - when itau = 1 and idel = 0, compute 1st temperature derivative
 - when itau = 2 and idel = 0, compute 2nd temperature derivative
 - when itau = 0 and idel = 1, compute 1st density derivative (actually the derivatives are with respect to the dimensionless quantities tau and delta)
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **phi00** – [double&, out] :: Ideal-gas part of the reduced Helmholtz energy (A/RT); derivatives (as specified by itau and idel) are multiplied by the corresponding power of tau or delta; i.e.,
 - when itau = 1, the quantity returned is tau*[d(PHI0)/d(tau)]
 - when itau = 2, the quantity returned is tau^2*[d^2(PHI0)/d(tau)^2]
 - when itau = 3, the quantity returned is tau^3*d^3(phi0cpp)/d(tau)^3 where tau=Tc/T and delta=D/Dc are evaluated for each component. Similarly, the delta derivatives (as specified by idel) are multiplied by the corresponding

power of delta (the derivatives usually appear with this factor and this approach neatly aEXPORT_CODE void CONVENTIONS a possible divide by zero).

```
void PHIDERVcpp(const Handle model_hdl, const int iderv, const double T, const double D, const
                std::vector<double> &z, std::vector<double> &dadn, std::vector<double>
                &dnadn, int &ierr, std::string &herr, const int herr_length = 255)
```

Calculate various derivatives required in the calculation of VLE for mixtures. Most of these are based on equations in the GERG-2004 document for natural gas, and are given below on the lines where the code corresponds directly to the equation in that document. Only the partials of alpha or alpha*n with respect to mole number are returned here. All others are stored in the PHIDR common block for access by subroutine SATGV.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **iderv** – [const int, in] :: Set to 1 for first order derivatives only (dadn and dnadn). Set to 2 for full calculations
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **dadn** – [std::vector<double>&, out] :: $n \cdot \text{partial}(\alpha) / \text{partial}(n_i)$ Eq. 7.16 in GERG-2004, 20 elements
- **dnadn** – [std::vector<double>&, out] :: $\text{partial}(n \cdot \alpha) / \text{partial}(n_i)$ Eq. 7.15 in GERG-2004, 20 elements
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: Error string length

```
void PHIKcpp(const Handle model_hdl, const int icomp, const int itau, const int idel, const double
             tau, const double delta, double &phi)
```

Compute reduced Helmholtz energy or a derivative as functions of dimensionless temperature and density.

The Helmholtz energy consists of ideal and residual (real-gas) terms; this routine calculates only the residual part.

This function computes pure component properties only; call PHIXcpp instead for mixtures.

The reducing parameters T_r and D_r are often, but not necessarily, equal to the critical temperature and density for pure fluids.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **icomp** – [const int, in] :: Pointer specifying component (1..nc)

- **itau** – [const int, in] :: Flag specifying the order of the temperature derivative
- **idel** – [const int, in] :: Flag specifying the order of the density derivative. When itau = 0 and idel = 0, compute A/RT. When itau = 0 and idel = 1, compute 1st density derivative. When itau = 1 and idel = 1, compute cross derivative. etc.
- **tau** – [const double, in] :: Dimensionless temperature (T_r/T)
- **delta** – [const double, in] :: Dimensionless density (D/D_r)
- **phi** – [double&, out] :: Residual (real-gas) part of the Helmholtz energy, or one of its derivatives (as specified by itau and idel), in reduced form (A/RT)

void **PHIMIXcpp**(const *Handle* model_hdl, const int i, const int j, const int itau, const int idel, const double tau, const double delta, const std::vector<double> &z, double &phi)

Compute reduced Helmholtz energy of mixing (or its derivatives) for the binary interaction of components i and j as a function of composition and dimensionless temperature and density for the mixture Helmholtz equation of state.

The Helmholtz energy consists of ideal-gas and residual (real-gas) terms. The residual term consists of ideal-solution and mixing terms. This routine calculates only the residual term.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **i** – [const int, in] :: Component i
- **j** – [const int, in] :: Component j
- **itau** – [const int, in] :: Flag specifying the order of the temperature derivative
- **idel** – [const int, in] :: Flag specifying the order of the density derivative. When itau = 0 and idel = 0, compute A_{mix}/RT . When itau = 0 and idel = 1, compute 1st density derivative. When itau = 1 and idel = 1, compute cross derivative. etc.
- **tau** – [const double, in] :: Dimensionless temperature (T_r/T)
- **delta** – [const double, in] :: Dimensionless density (D/D_r)
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **phi** – [double&, out] :: Mixture interaction (excess) part of the Helmholtz energy, or one of its derivatives (as specified by itau and idel), in reduced form (A_{mix}/RT)

void **PHIMXcpp**(const *Handle* model_hdl, const int itau, const int idel, const double tau, const double delta, const std::vector<double> &z, double &phi)

Compute reduced Helmholtz energy or its derivative as functions of dimensionless temperature and density for the mixture Helmholtz equation of state.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **itau** – [const int, in] :: Flag specifying the order of the temperature derivative

- **idel** – [const int, in] :: Flag specifying the order of the density derivative. When itau = 0 and idel = 0, compute A/RT. When itau = 0 and idel = 1, compute 1st density derivative. When itau = 1 and idel = 1, compute cross derivative. etc.
- **tau** – [const double, in] :: Dimensionless temperature (T_r/T)
- **delta** – [const double, in] :: Dimensionless density (D/D_r)
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **phi** – [double&, out] :: Residual (real-gas) part of the Helmholtz energy, or one of its derivatives (as specified by itau and idel), in reduced form (A/RT)

void **PHIXcpp**(const *Handle* model_hdl, const int itau, const int idel, const double tau, const double delta, const std::vector<double> &z, double &phixx)

Compute reduced Helmholtz energy or a derivative as functions of dimensionless temperature and density by calling the appropriate mixture model. The Helmholtz energy consists of ideal-gas and residual (real-gas) terms. The residual term consists of ideal-solution and mixing terms. This routine calculates only the residual term.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **itau** – [const int, in] :: Flag specifying the order of the temperature derivative
- **idel** – [const int, in] :: Flag specifying the order of the density derivative
 - When itau = 0 and idel = 0, compute A/RT.
 - When itau = 0 and idel = 1, compute 1st density derivative.
 - When itau = 1 and idel = 1, compute cross derivative. etc.
- **tau** – [const double, in] :: Dimensionless temperature (T_r/T)
- **delta** – [const double, in] :: Dimensionless density (D/D_r)
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **phixx** – [double&, out] :: Residual (real-gas) part of the Helmholtz energy, or one of its derivatives (as specified by itau and idel), in reduced form (A/RT). Value is multiplied by tau and delta to the powers of itau and idel, respectively.

void **PREOScpp**(const *Handle* model_hdl, int &i)

Turn on or off the use of the PR cubic equation. Should be called after calling SETUP.

If i=-1, then i is returned with the current status of the PR EOS. A value of zero indicates that it is not in use. When in use, a 2 or 3 will be returned, depending on which option was previously selected.

Flags flags

-1 Query: returns current status in i 0 Use full equation of state (Peng-Robinson off) 1 Use

full equation of state with Peng-Robinson for sat. conditions (not currently working) **2** Use Peng-Robinson equation for all calculations **3** Peng-Robinson with translation term deactivated

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **i** – [int&, in/out] :: Flag specifying use of PR

void **PRESScpp**(const *Handle* model_hdl, const double T, const double D, const std::vector<double> &z, double &P)

Compute pressure as a function of temperature, density, and composition. See warning in THERMdll or ALLPROPSdll.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **P** – [double&, out] :: Pressure [kPa]

void **PSATKcpp**(const *Handle* model_hdl, const int icomp, const double T, double &P, int &ierr, std::string &herr, const int herr_length = 255)

Compute pure fluid vapor or liquid pressures with the appropriate ancillary equation.

Flags flags

0 Successful **121** Temperature greater than critical point temperature

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **icomp** – [const int, in] :: Component i. For liquid pressure equations (pseudo-pure fluids only), send -i.
- **T** – [const double, in] :: Temperature [K]
- **P** – [double&, out] :: Vapor or liquid pressure [kPa]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

–

```
void PSFL1cpp(const Handle model_hdl, const double P, const double s, const std::vector<double>
              &z, const int kph, double &T, double &D, int &ierr, std::string &herr, const int
              herr_length = 255)
```

Iterate for single-phase temperature and density as a function of pressure, entropy, and composition.

Flags flags

1 Liquid phase **2** Vapor phase

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **P** – [const double, in] :: Pressure [kPa]
- **s** – [const double, in] :: Entropy [J/mol-K]
- **z** – [const std::vector<double>&, in] :: Bulk composition (vector of mole fractions), maximum of 20 elements
- **kph** – [const int, in] :: Phase flag
- **T** – [double&, out] :: Temperature [K]
- **D** – [double&, out] :: Density [mol/L]
- **ierr** – [int&, out] :: Error code
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void PUREFLDcpp(const Handle model_hdl, const int icomp)
```

Change the standard mixture setup so that the properties of one fluid can be calculated as if SETUP had been called for a pure fluid. Calling this routine will disable all mixture calculations. To reset the mixture setup, call this routine with icomp=0.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **icomp** – [const int, in] :: Fluid number in a mixture to use as a pure fluid (set to zero to reset)

```
void QMASScpp(const Handle model_hdl, const double qmol, const std::vector<double> &x1, const
              std::vector<double> &xv, double &qkg, std::vector<double> &xlkg,
              std::vector<double> &xvkg, double &wliq, double &wvap, int &ierr, std::string
              &herr, const int herr_length = 255)
```

Convert quality and composition on a molar basis to a mass basis.

Flags flags

0 All inputs within limits **-19** Input qmol < 0 or > 1

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **qmol** – [const double, in] :: Molar quality (moles of vapor/total moles) qmol = 0 indicates saturated liquid qmol = 1 indicates saturated vapor $0 < \text{qmol} < 1$ indicates a two-phase state qmol < 0 or qmol > 1 are not allowed and will result in warning
- **xl** – [const std::vector<double>&, in] :: Composition of liquid phase (vector of mole fractions), maximum of 20 elements
- **xv** – [const std::vector<double>&, in] :: Composition of vapor phase (vector of mole fractions), maximum of 20 elements
- **qkg** – [double&, out] :: Quality on mass basis (mass of vapor/total mass) [-]
- **xlkg** – [std::vector<double>&, out] :: Mass composition of liquid phase (vector of mass fractions), maximum of 20 elements
- **xvkg** – [std::vector<double>&, out] :: Mass composition of vapor phase (vector of mass fractions), maximum of 20 elements
- **wliq** – [double&, out] :: Molar mass of liquid phase [g/mol]
- **wvap** – [double&, out] :: Molar mass of vapor phase [g/mol]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: Error string length (default: 255)

```
void QMOLEcpp(const Handle model_hdl, const double qkg, const std::vector<double> &xlkg, const  
std::vector<double> &xvkg, double &qmol, std::vector<double> &xl,  
std::vector<double> &xv, double &wliq, double &wvap, int &ierr, std::string &herr,  
const int herr_length = 255)
```

Convert quality and composition on a mass basis to a molar basis.

Flags flags

- 0** All inputs within limits **-19** Input qkg < 0 or > 1

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **qkg** – [const double, in] :: Quality on mass basis (mass of vapor/total mass) qkg = 0 indicates saturated liquid qkg = 1 indicates a saturated vapor $0 < \text{qkg} < 1$ indicates a two-phase state qkg < 0 or qkg > 1 are not allowed and will result in warning
- **xlkg** – [const std::vector<double>&, in] :: Mass composition of liquid phase (vector of mass fractions), maximum of 20 elements

- **xvkg** – [const std::vector<double>&, in] :: Mass composition of vapor phase (vector of mass fractions), maximum of 20 elements
- **qmol** – [double&, out] :: Quality on molar basis (moles of vapor/total moles) [-]
- **xl** – [std::vector<double>&, out] :: Molar composition of liquid phase (vector of mole fractions), maximum of 20 elements
- **xv** – [std::vector<double>&, out] :: Molar composition of vapor phase (vector of mole fractions), maximum of 20 elements
- **wliq** – [double&, out] :: Molar mass of liquid phase [g/mol]
- **wvap** – [double&, out] :: Molar mass of vapor phase [g/mol]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: Error string length (default: 255)

void **RDHMXcpp**(const *Handle* model_hdl, const int ix, const int icmp, const int icmp2, const std::vector<double> &z, double &Tred, double &Dred, int &ierr, std::string &herr, const int herr_length = 255)

Returns reducing parameters and their derivatives associated with the mixture Helmholtz EOS; these are used to calculate the ‘tau’ and ‘del’ that are the independent variables in the EOS.

Flags flags

0 Successful **301** Mixing rule not found for i,j **191** Derivative not available

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **ix** – [const int, in] :: Flag specifying the order of the composition derivative to calculate. When ix = 0, compute T(red) and D(red). For icmp2 = 0: when ix = 1, compute 1st derivative with respect to z(icmp) or z(icmp2); when ix = 2, compute 2nd derivative with respect to z(icmp) or z(icmp2). For icmp != 0 and icmp2 != 0: when ix = 11, compute cross derivative with respect to z(icmp) and z(icmp2).
- **icmp** – [const int, in] :: Component number for which derivative will be calculated
- **icmp2** – [const int, in] :: Second component number for which derivative will be calculated
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **Tred** – [double&, out] :: Reducing temperature [K] or derivative

- **Dred** – [double&, out] :: Reducing molar density [mol/L] or derivative of reducing volume [L/mol] (ix = 0 - Dc; ix = 1 - dVc/dxi; ix = 2 - d^2Vc/dxi^2; ix = 11 - d^2Vc/dxidxj)
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **REDXcpp**(const *Handle* model_hdl, const std::vector<double> &z, double &Tred, double &Dred)

Returns the reducing parameters associated with mixture EOS; used to calculate the tau and delta, which are the independent variables in the EOS.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **Tred** – [double&, out] :: Reducing temperature [K]
- **Dred** – [double&, out] :: Reducing molar density [mol/L]

void **RESIDUALcpp**(const *Handle* model_hdl, const double T, const double D, const std::vector<double> &z, double &Pr, double &er, double &hr, double &sr, double &Cvr, double &Cpr, double &ar, double &gr)

Compute the residual quantities as a function of temperature, density, and composition (where the residual is the total property minus the ideal gas portion). This routine is the same as THERM2, except it only calculates the residual portions at any temperature and density.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **Pr** – [double&, out] :: Residual pressure [kPa]
- **er** – [double&, out] :: Residual internal energy [J/mol]
- **hr** – [double&, out] :: Residual enthalpy [J/mol]
- **sr** – [double&, out] :: Residual entropy [J/mol-K]
- **Cvr** – [double&, out] :: Residual isochoric heat capacity [J/mol-K]
- **Cpr** – [double&, out] :: Residual isobaric heat capacity [J/mol-K]
- **ar** – [double&, out] :: Residual Helmholtz energy [J/mol]

- **gr** – [double&, out] :: Residual Gibbs free energy [J/mol]

void **RIEMcpp**(const *Handle* model_hdl, const double T, const double D, const std::vector<double> &z, double &riemc)

RIEM is the thermodynamic curvature in cubic nanometers/molecule. It has the magnitude of the correlation volume, is negative for attractive interactions, and positive for repulsive interactions, except when its magnitude gets smaller than the molecular volume.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **riemc** – [double&, out] :: RIEM [cubic nanometers/molecule]

void **RMIX2cpp**(const *Handle* model_hdl, const std::vector<double> &z, double &Rgas)

Mimic RMIX but return the gas constant as a parameter for use in the DLL.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **Rgas** – [double&, out] :: Gas constant [J/(mol-K)]

void **SATDcpp**(const *Handle* model_hdl, const double D, const std::vector<double> &z, const int kph, int &kr, double &T, double &P, double &Dl, double &Dv, std::vector<double> &x, std::vector<double> &y, int &ierr, std::string &herr, const int herr_length = 255)

Iterate for temperature and pressure given density along the saturation boundary (including the sublimation and melting lines) and the composition.

Either (Dl,x) or (Dv,y) will correspond to the input state with the other pair corresponding to the other phase in equilibrium with the input state.

The flag kph is for use only with water at densities near the triple point (between 0 and 4 C).

Flags flags

0, **1** Return upper root **-1** Return middle root **3** Return melting line

Flags flags

1 Input state is liquid in equilibrium with vapor **2** Input state is vapor in equilibrium with liquid **3** Input state is liquid in equilibrium with solid (only for pure fluids) **4** Input state is vapor in equilibrium with solid (only for pure fluids)

Flags flags

0 Successful **2** D>Dtrp of the liquid **3** D<Dtrp of the vapor **160** SATD did not converge

(See subroutine LIMITX for other possible error numbers.)

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **kph** – [const int, in] :: Flag specifying desired root for multi-valued inputs (typically only water)
- **kr** – [int&, out] :: Phase flag
- **T** – [double&, out] :: Temperature [K]
- **P** – [double&, out] :: Pressure [kPa]
- **Dl** – [double&, out] :: Molar density of saturated liquid [mol/L]
- **Dv** – [double&, out] :: Molar density of saturated vapor [mol/L]
- **x** – [std::vector<double>&, out] :: Liquid phase composition (vector of mole fractions)
- **y** – [std::vector<double>&, out] :: Vapor phase composition (vector of mole fractions)
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **SATEcpp**(const *Handle* model_hdl, const double e, const std::vector<double> &z, const int kph, int &nroot, int &k1, double &T1, double &P1, double &D1, int &k2, double &T2, double &P2, double &D2, int &ierr, std::string &herr, const int herr_length = 255)

Iterate for temperature, pressure, and density given energy along the saturation boundary and the composition.

(See SATHcpp for the description of outputs.)

Flags flags

0 Return all roots along the liquid-vapor line **1** Return only the liquid VLE root **2** Return only the vapor VLE roots **3** Return liquid SLE root (melting line) **4** Return vapor SVE root (sublimation line)

Flags flags

0 Successful **54** $e < e_{min}$ **55** $e > e_{max}$ **56** $e > e_{trp}$ (for sublimation inputs) **181** SATE did not converge for one of the roots

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **e** – [const double, in] :: Molar energy [J/mol]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **kph** – [const int, in] :: Flag specifying desired root
- **nroot** – [int&, out] :: Number of roots. Value is set to one for kph=1,3,4 if ierr=0
- **k1** – [int&, out] :: Phase of first root (1-liquid, 2-vapor, 3-melt, 4-subl)
- **T1** – [double&, out] :: Temperature of first root [K]
- **P1** – [double&, out] :: Pressure of first root [kPa]
- **D1** – [double&, out] :: Molar density of first root [mol/L]
- **k2** – [int&, out] :: Phase of second root (1-liquid, 2-vapor, 3-melt, 4-subl)
- **T2** – [double&, out] :: Temperature of second root [K]
- **P2** – [double&, out] :: Pressure of second root [kPa]
- **D2** – [double&, out] :: Molar density of second root [mol/L]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **SATESTcpp**(const *Handle* model_hdl, const int iFlash, double &T, double &P, const std::vector<double> &z, std::vector<double> &x, std::vector<double> &y, int &ierr, std::string &herr, const int herr_length = 255)

Estimate temperature, pressure, and compositions to be used as initial guesses to SATTP.

Flags flags

0 Flash calculation (T and P known) **1** T and x known, P and y returned **2** T and y known, P and x returned **3** P and x known, T and y returned **4** P and y known, T and x returned. If this value is negative, the retrograde point will be returned.

Flags flags

0 Successful **999** Unsuccessful

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **iFlash** – [const int, in] :: Phase flag
- **T** – [double&, in/out] :: Temperature [K] (input or output)
- **P** – [double&, in/out] :: Pressure [kPa] (input or output)

- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements. The composition for the known x or y array should be sent in this z array, not in the output arrays shown below.
- **x** – [std::vector<double>&, out] :: Liquid phase composition (vector of mole fractions)
- **y** – [std::vector<double>&, out] :: Vapor phase composition (vector of mole fractions)
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void SATGUESScpp(const Handle model_hdl, const int kph, const int iprop, const std::vector<double>
    &x, double &T, double &P, double &D, double &h, double &s, double &Dy,
    std::vector<double> &y, int &ierr, std::string &herr, const int herr_length = 255)
```

For a pure fluid, call the ancillary equations to obtain close estimates for the saturation boundaries. The difference between these values and those from SATT, SATP, etc., depend on how well the ancillary equation was fitted, but generally they are within 0.1%, except for the saturation densities within several degrees of the critical temperature.

For a mixture, calculate approximate values from the spline curves for the saturation boundary. Subroutine SATSPLN must be called in order for this to work.

The input property should be placed in the corresponding variable for T, P, D, h, or s. Inputs of h and s only work for mixtures when SATSPLN has been called.

Flags flags

1 Temperature **2** Pressure **3** Density **4** Enthalpy **5** Entropy **6** Energy (currently only working for pure fluids) **11 to 15** 1st derivative of the property (for iprop=10) returned in Dy with respect to density (only for iFlag=0) **21 to 25** 2nd derivative of the property (for iprop=20) returned in Dy with respect to density (only for iFlag=0) **101** Check if the x array is identical to those sent to SATSPLN (only for iFlag=0) (for negative values of 1 to 5,

find the location of zero slope of the property with respect to D) (only for iFlag=0)

Flags flags

0 Use default and best methods, generally ancillary equations for pure fluids and splines (calculated from call to SATSPLN) for mixtures **11** Use Rackett technique to get density from T and P (value of kph and iprop ignored) **12** Use initial guess equations of Lemmon

Flags flags

0 Successful **331** Splines not available for saturation calculations **332** Initialize variable d721 first before calling SATGUESS **-311** Compositions not identical to that used in the call to SATSPLN

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle

- **kph** – [const int, in] :: Input phase; 1-liquid, 2-vapor. When maximum in the property does not occur near the critical point, then kph=1 returns the root at the higher density and kph=2 returns the root at the lower density.
- **iprop** – [const int, in] :: Input property
- **x** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **T** – [double&, in/out] :: Temperature [K]
- **P** – [double&, in/out] :: Pressure [kPa]
- **D** – [double&, in/out] :: Density [mol/L]
- **h** – [double&, out] :: Enthalpy or energy [J/mol] (not returned for a pure fluid)
- **s** – [double&, out] :: Entropy [J/mol-K] (not returned for a pure fluid)
- **Dy** – [double&, out] :: Equilibrium phase density [mol/L]
- **y** – [std::vector<double>&, out] :: Equilibrium phase composition (vector of mole fractions). (h, s, and y are only returned when splines are used to calculate values.)
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void SATGVcpp(const Handle model_hdl, double &T, double &P, const std::vector<double> &z,
              const double vf, const double b, const int ipv, const int ityp, const int isp, double
              &Dx, double &Dy, std::vector<double> &x, std::vector<double> &y, int &ierr,
              std::string &herr, const int herr_length = 255)
```

Calculates the bubble or dew point state with the entropy or density method of GV. The calculation method is similar to the volume based algorithm of GERG. The cricondenbar and criconden-therm are estimated with the method in Michelsen, Saturation Point Calculations, Fluid Phase Equilibria, 23:181, 1985.

Equations to be solved simultaneously are

Pressure based:

```
f(1:n) - LOG(y/x)-LOG((fxi/nxi)/(fyi/nyi))=0
f(n+1) - SUM(y(i)-x(i))=0
f(n+2) - b/binput-1=0, where b = P, T, D, or s
```

Volume based:

```
f(1:n) - LOG(y/x)-LOG((fxi/nxi)/(fyi/nyi))=0
f(n+1) - SUM(y(i)-x(i))=0
f(n+2) - py=px
f(n+3) - b/binput-1=0, where b = P, T, D, or s
```

Variables:

```
1 to nc - LOG(k(i))
nc+1    - LOG(T)
nc+2    - LOG(P) or LOG(Dx)
nc+3    - LOG(Dy)
```

Initial guesses must be sent in all variables (T, P, Dx, Dy, x, and y) unless isp=1.

Flags flags

vf=0, ityp=0 or 1 Dew phase inputs, state in equilibrium returned in Dy and y **vf=1, ityp=0 or 1** Liquid phase inputs, state in equilibrium returned in Dx and x **vf=0, ityp=6** Inputs are returned in Dx and the x array; outputs are returned in Dy and the y array **vf=1, ityp=6** Inputs are returned in Dy and the y array; outputs are returned in Dx and the x array

Flags flags

1 Pressure based **2** Volume based

Flags flags

0 Given P, calculate T **1** Given T, calculate P **2** Cricondentherm condition, calculate T,P (ipv=1 only) **3** Cricondenbar condition, calculate T,P (ipv=1 only) **5** Given entropy, calculate T,P **6** Given density, calculate T,P

Flags flags

0 Successful **2** Input D<=0 **151** No convergence **172** vf<0 or vf>1 **191** Derivatives are not available in PR or RDXHMX **200** Density out of range **321** Trivial solution

(See subroutine LIMITX for other possible error numbers.)

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [double&, inout] :: Temperature [K]
- **P** – [double&, inout] :: Pressure [kPa]
- **z** – [const std::vector<double>&, in] :: Overall composition (vector of mole fractions), maximum of 20 elements
- **vf** – [const double, in] :: Vapor fraction (0<=vf<=1; the input value of density can be in either state and does not affect the outputs in Dx, Dy, x, and y)
- **b** – [const double, in] :: Input value, either entropy [J/mol-K] or density [mol/L]
- **ipv** – [const int, in] :: Pressure or volume based algorithm
- **ityp** – [const int, in] :: Input values
- **isp** – [const int, in] :: Use values from Splines as initial guesses if set to 1
- **Dx** – [double&, inout] :: Density of x phase [mol/L]
- **Dy** – [double&, inout] :: Density of y phase [mol/L]

- **x** – [std::vector<double>&, inout] :: Composition of the x array (vector of mole fractions)
- **y** – [std::vector<double>&, inout] :: Composition of the y array (vector of mole fractions)
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **SATHcpp**(const *Handle* model_hdl, const double h, const std::vector<double> &z, const int kph, int &nroot, int &k1, double &T1, double &P1, double &D1, int &k2, double &T2, double &P2, double &D2, int &ierr, std::string &herr, const int herr_length = 255)

Iterate for temperature, pressure, and density given enthalpy along the saturation boundary and the composition.

The second root is always set as the root in the vapor at temperatures below the maximum enthalpy on the vapor saturation line. If kph is set to 2, and only one root is found in the vapor (this occurs when $h < h_{crit}$) the state point will be placed in k2, T2, P2, D2. If kph=0 and this situation occurred, the first root (k1, T1, P1, D1) would be in the liquid (k1=1, k2=2).

Flags flags

0 Return all roots along the liquid-vapor line **1** Return only liquid VLE root **2** Return only vapor VLE roots **3** Return liquid SLE root (melting line) **4** Return vapor SVE root (sublimation line)

kph = 3,4 presently working only for pure components

Flags flags

0 Successful **54** $h < h_{min}$ **55** $h > h_{max}$ **56** $h > h_{trp}$ (for sublimation inputs) **181** SATH did not converge for one of the roots

(See subroutine LIMITX for other possible error numbers.)

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **h** – [const double, in] :: Molar enthalpy [J/mol]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **kph** – [const int, in] :: Flag specifying desired root
- **nroot** – [int&, out] :: Number of roots. Value is set to one for kph=1,3,4 if ierr=0
- **k1** – [int&, out] :: Phase of first root (1-liquid, 2-vapor, 3-melt, 4-subl).
- **T1** – [double&, out] :: Temperature of first root [K]
- **P1** – [double&, out] :: Pressure of first root [kPa]

- **D1** – [double&, out] :: Molar density of first root [mol/L]
- **k2** – [int&, out] :: Phase of second root (1-liquid, 2-vapor, 3-melt, 4-subl)
- **T2** – [double&, out] :: Temperature of second root [K]
- **P2** – [double&, out] :: Pressure of second root [kPa]
- **D2** – [double&, out] :: Molar density of second root [mol/L]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **SATPcpp**(const *Handle* model_hdl, const double P, const std::vector<double> &z, const int kph, double &T, double &Dl, double &Dv, std::vector<double> &x, std::vector<double> &y, int &ierr, std::string &herr, const int herr_length = 255)

Iterate for saturated liquid and vapor states given pressure and the composition of one phase.

If T is negative, all other variables are used as initial guesses at ABS(T).

Flags flags

1 Input z is liquid composition (bubble point) **2** Input z is vapor composition (dew point) **3** Input z is liquid composition (freezing point) **4** Input z is vapor composition (sublimation point)

Flags flags

0 Successful **141** $P > P_{crit}$ **144** Pure fluid iteration did not converge

(See subroutine LIMITX for other possible error numbers.)

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **P** – [const double, in] :: Pressure [kPa]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions) (phase specified by kph), maximum of 20 elements
- **kph** – [const int, in] :: Phase flag
- **T** – [double&, inout] :: Temperature [K] (input or output)
- **D1** – [double&, out] :: Molar density of saturated liquid [mol/L]
- **Dv** – [double&, out] :: Molar density of saturated vapor [mol/L]. For a pseudo pure fluid, the density of the equilibrium phase is not returned. Call SATPcpp twice, once with kph=1 to get Tliq and Dl, and once with kph=2 to get Tvap and Dv.
- **x** – [std::vector<double>&, out] :: Liquid phase composition (vector of mole fractions)

- **y** – [std::vector<double>&, out] :: Vapor phase composition (vector of mole fractions)
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void SATScpp(const Handle model_hdl, const double s, const std::vector<double> &z, const int kph,
             int &nroot, int &k1, double &T1, double &P1, double &D1, int &k2, double &T2,
             double &P2, double &D2, int &k3, double &T3, double &P3, double &D3, int &ierr,
             std::string &herr, const int herr_length = 255)
```

Iterate for temperature, pressure, and density given entropy along the saturation boundary and the composition.

The second root is always set as the root in the vapor at temperatures below the maximum entropy on the vapor saturation line. If kph is set to 2, and only one root is found in the vapor (this occurs when $s < s_{crit}$) the state point will be placed in k2, T2, P2, D2. If kph=0 and this situation occurred, the first root (k1, T1, P1, D1) would be in the liquid (k1=1, k2=2).

The third root is the root with the lowest temperature. For fluids with multiple roots, when only one root is found in the vapor phase (this happens only at very low temperatures past the region where three roots are located), the value of the root is still placed in k3, T3, P3, D3. For fluids that never have more than one root (when there is no maximum entropy along the saturated vapor line), the value of the root is always placed in k1, T1, P1, D1.

Flags flags

0 Return all roots along the liquid-vapor line **1** Return only liquid VLE root **2** Return only vapor VLE roots **3** Return liquid SLE root (melting line) **4** Return vapor SVE root (sublimation line)

kph = 3,4 presently working only for pure components

Flags flags

0 Successful **66** $s < s_{min}$ **67** $s > s_{max}$ **68** $s > s_{trp}$ (for sublimation inputs) **192** SATS did not converge for one or more roots

(See subroutine LIMITX for other possible error numbers.)

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **s** – [const double, in] :: Molar entropy [J/mol-K]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **kph** – [const int, in] :: Flag specifying desired root
- **nroot** – [int&, out] :: Number of roots. Set to one for kph=1,3,4 if ierr=0

- **k1** – [int&, out] :: Phase of first root (1-liquid, 2-vapor, 3-melt, 4-subl)
- **T1** – [double&, out] :: Temperature of first root [K]
- **P1** – [double&, out] :: Pressure of first root [kPa]
- **D1** – [double&, out] :: Molar density of first root [mol/L]
- **k2** – [int&, out] :: Phase of second root (1-liquid, 2-vapor, 3-melt, 4-subl)
- **T2** – [double&, out] :: Temperature of second root [K]
- **P2** – [double&, out] :: Pressure of second root [kPa]
- **D2** – [double&, out] :: Molar density of second root [mol/L]
- **k3** – [int&, out] :: Phase of third root (1-liquid, 2-vapor, 3-melt, 4-subl)
- **T3** – [double&, out] :: Temperature of third root [K]
- **P3** – [double&, out] :: Pressure of third root [kPa]
- **D3** – [double&, out] :: Molar density of third root [mol/L]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **SATSPLNcpp**(const *Handle* model_hdl, const std::vector<double> &z, int &ierr, std::string &herr, const int herr_length = 255)

Calculates the phase boundary of a mixture at a given composition, along with the critical point, cricondentherm, and cricondenbar.

Flags flags

0 Successful **355** Saturation routine failed

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **SATCcpp**(const *Handle* model_hdl, double &T, const std::vector<double> &z, const int kph, double &P, double &DI, double &Dv, std::vector<double> &x, std::vector<double> &y, int &ierr, std::string &herr, const int herr_length = 255)

Iterate for saturated liquid and vapor states given temperature and the composition of one phase.

If T is negative, all other variables are used as initial guesses at ABS(T).

Flags flags

1 Input z is liquid composition (bubble point) **-1** Force calculation in the liquid phase even if $T < T_{\text{trp}}$

****2**** Input z is vapor composition (dew point)

**** -2**** Force calculation in the vapor phase even if $T < T_{\text{trp}}$

****3**** Input z is liquid composition along the freezing line (melting line)

****4**** Input z is vapor composition along the sublimation line

Flags flags

****0**** Successful

****121**** $T > T_{\text{crit}}$ **124** Pure fluid iteration did not converge

(See subroutine LIMITX for other possible error numbers.)

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [double&, in] :: Temperature [K]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions) (phase specified by kph), maximum of 20 elements
- **kph** – [const int, in] :: Phase flag
- **P** – [double&, out] :: Pressure [kPa]
- **Dl** – [double&, out] :: Molar density of saturated liquid [mol/L]
- **Dv** – [double&, out] :: Molar density of saturated vapor [mol/L]. For a pseudo pure fluid, the density of the equilibrium phase is not returned. Call SATTcpp twice, once with kph=1 to get Pliq and Dl, and once with kph=2 to get Pvap and Dv.
- **x** – [std::vector<double>&, out] :: Liquid phase composition (vector of mole fractions)
- **y** – [std::vector<double>&, out] :: Vapor phase composition (vector of mole fractions)
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void SATTPcpp(const Handle model_hdl, double &T, double &P, const std::vector<double> &z,  
             const int iFlash, const int iGuess, double &D, double &Dl, double &Dv,  
             std::vector<double> &x, std::vector<double> &y, double &q, int &ierr, std::string  
             &herr, const int herr_length = 255)
```

Calculate saturation properties for bubble, dew, or 2-phase states with the use of analytical derivatives of the Helmholtz energy with respect to composition.

Flags flags

0 Flash calculation (T and P known) **1** T and x known, P and y returned **2** T and y known, P and x returned **3** P and x known, T and y returned **4** P and y known, T and x returned

If this value is negative, the retrograde point will be returned.

Flags flags

0 No initial guess **1** Parameters Dl, Dv, x, and y are used as initial guesses; if Dl and Dv are zero, densities are obtained from TPRHO **2** Use inputs rather than spline values (if splines have been calculated)

Flags flags

0 Successful **121** T>Tmax (maxcondentherm) **141** P>Pmax (maxcondenbar) **151** Iteration failed **156** Probable Type III mixture with no liquid solution **159** Wrong input value for iFlash
(See subroutine LIMITX for other possible error numbers.)

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [double&, inout] :: Temperature [K] (input or output)
- **P** – [double&, inout] :: Pressure [kPa] (input or output)
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **iFlash** – [const int, in] :: Phase flag
- **iGuess** – [const int, in] :: Initial guess flag
- **D** – [double&, inout] :: Overall density [mol/L]
- **Dl** – [double&, inout] :: Molar density of saturated liquid [mol/L]
- **Dv** – [double&, inout] :: Molar density of saturated vapor [mol/L]
- **x** – [std::vector<double>&, inout] :: Liquid phase composition (vector of mole fractions)
- **y** – [std::vector<double>&, inout] :: Vapor phase composition (vector of mole fractions)
- **q** – [double&, out] :: Quality
- **ierr** – [int&, out] :: Error flag

- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **SUBLPcpp**(const *Handle* model_hdl, const double P, const std::vector<double> &z, double &T, int &ierr, std::string &herr, const int herr_length = 255)

Compute the sublimation line temperature as a function of pressure and composition.

Flags flags

0 Successful **-4** Pressure above triple point pressure **501** No equation available

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **P** – [const double, in] :: Sublimation line pressure [kPa]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **T** – [double&, out] :: Temperature [K]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **SUBLTcpp**(const *Handle* model_hdl, const double T, const std::vector<double> &z, double &P, int &ierr, std::string &herr, const int herr_length = 255)

Compute the sublimation line pressure as a function of temperature and composition.

Flags flags

0 Successful **501** No equation available

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **P** – [double&, out] :: Sublimation line pressure [kPa]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **SETAGAcpp**(const *Handle* model_hdl, int &ierr, std::string &herr, const int herr_length = 255)
 Set up working arrays for use with AGA8 equation of state.

Flags flags

0 Successful **108** Error (e.g. fluid not found)

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void SETFLUIDScpp (Handle &model_hdl, const std::string &hFld, int &ierr, const
bool reuse_default_handle=false, const std::string &ideal_gas_model_params=R"({
"model_type": ")" + kDefaultIdealGasModelType + R"(" })", const std::string
&residual_model_params=R"({ "model_type": ")" + kDefaultResidualModelType + R"("
}))")
```

Call the SETUPdll routine without the need to pass ncomp, hrf, hFmix, or herr, or to declare the length of hFld as 255 or 10000 bytes long. For a pure fluid, fluid_names simply contain the names of the fluid files (with a path if needed). For a mixture, it contains the names of the constituents in the mixture separated by a |, a semicolon, or an asterisk. To load a predefined mixture, call the SETMIXTUREdll function (which must return the composition array and thus cannot be included here). If it is necessary to set the reference state, call SETUPdll instead. If ierr comes back non-zero, call the ERRMSGdll routine to obtain it.

Examples:

```
SETFLUIDScpp(model_hdl, "ARGON", ierr);           // load argon as a pure fluid
SETFLUIDScpp(model_hdl, "FLUIDS/NITROGEN.FLD|FLUIDS/ARGON.FLD|FLUIDS/OXYGEN.
↳FLD|",                                           // for the air mixture, but_
ierr);                                           // giving a path
SETFLUIDScpp(model_hdl, "AIR.PPF", ierr);         // load the air mixture, but_
↳read from the                                // pseudo-pure file;_
properties will be                             // slightly different from the_
↳*.mix file                                    // since they are different_
↳models
SETFLUIDScpp(model_hdl, "methane * ethane * propane * butane", ierr);
```

Flags flags

0 Successful (Values are identical to SETUP; a 109 is returned if the number of fluids in hfld is less than icomp.)

Parameters

- **model_hdl** – [uint64_t&, out] :: Model handle
- **hFld** – [const string&, in] :: String of any character length containing the fluid file names
- **ierr** – [int&, out] :: Error flag
- **reuse_default_handle** – [const bool, in] :: Reuse default, legacy model handle (default: false)
- **ideal_gas_model_params** – [const string&, in] :: JSON string of ideal gas model type and params
- **residual_model_params** – [const string&, in] :: JSON string of residual model type and params

```
void SETKTVcpp(const Handle model_hdl, const int icomp, const int jcomp, const std::string
               &hmodij, const std::vector<double> &fij, const std::string &hFmix, int &ierr,
               std::string &herr, const int hmodij_length = 3, const int hFmix_length = 255, const
               int herr_length = 255)
```

Set mixture model and/or parameters.

This subroutine must be called after SETUP, but before any call to SETREF (for cases where energy, enthalpy, entropy, Gibbs energy, or the Helmholtz energy are required); it need not be called at all if the default mixture parameters (those read in by SETUP) are to be used.

The component numbers icomp and jcomp must match the order that is found in the HMX.BNC file for each binary pair, or, in the case where no interaction parameters are available in the HMX.BNC file, icomp and jcomp must be in the same order as was used in the call to SETUP. If the numbers in these two integers are backwards, an error number and message will be returned, and nothing will be changed. In this situation, switch the numbers and call this routine again.

Kunz-Wagner model (KW0)	Lemmon-Jacobsen model (LJ6)
fij(1) = betaT	fij(1) = zeta
fij(2) = gammaT	fij(2) = xi
fij(3) = betaV	fij(3) = Fij
fij(4) = gammaV	fij(4) = beta
fij(5) = Fij	fij(5) = gamma
fij(6) = 'not used'	fij(6) = 'not used'

Flags flags

0 Successful **111** Error in opening mixture file **902** Routine not valid for a pure fluid **903** Illegal i,j specification (i=j or i>nc or j>nc) **904** Order of fluids is backwards from that in HMX.BNC

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **icomp** – [const int, in] :: Component i
- **jcomp** – [const int, in] :: Component j
- **hmodij** – [const std::string&, in] :: Mixing rule for the binary pair i,j (e.g. LJ6, KW0, XR0, or LIN) (character*3). If hmodij is ‘RST’, reset all pairs to values from the original call to SETUP (all other inputs are ignored).
- **fij** – [const std::vector<double>&, in] :: Binary mixture parameters (array of dimension nmixpar; currently nmixpar is set to 6). The parameters will vary depending on hmodij.
- **hFmix** – [const std::string&, in] :: No longer used. Info from previous versions: File name (character*255) containing generalized parameters for the binary mixture model; this will usually be the same as the corresponding input to SETUP (e.g., ‘HMX.BNC’)
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **hmodij_length** – [const int, in] :: length of variable hmodij (default: 3)
- **hFmix_length** – [const int, in] :: length of variable hFmix (default: 255)
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void SETMIXcpp (Handle &model_hdl, const std::string &hMixNme, const std::string
&hFmix, const std::string &hrf, int &nc, std::string &hFiles, std::vector<
double > &z, int &ierr, std::string &herr, const bool
reuse_default_handle=false, const std::string &ideal_gas_model_params=R"({
"model_type": ")" + kDefaultIdealGasModelType + R"(" })", const std::string
&residual_model_params=R"({ "model_type": ")" + kDefaultResidualModelType + R"("
})")
```

Open a mixture file (e.g., R410A.mix) and read constituents and mole fractions.

Flags flags

0 Successful **101** Error in opening file **-102** Mixture file contains mixing parameters **805** Sum of compositions not equal to one

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle

- **hMixNme** – [const std::string&, in] :: Mixture file name
- **hFmix** – [const std::string&, in] :: File containing mixture coefficients
- **hrf** – [const std::string&, in] :: Reference state (character*3); See subroutine SETUP for specifics.
- **ncc** – [int&, out] :: Number of fluids in mixture
- **hFiles** – [std::string&, out] :: Array of file names specifying mixture components that were used to call setup
- **z** – [std::vector<double>&, out] :: Array of mole fractions for the specified mixture
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **reuse_default_handle** – [const bool, in] :: Reuse default, legacy model handle (default: false)
- **ideal_gas_model_params** – [const string&, in] :: JSON string of ideal gas model type and params
- **residual_model_params** – [const string&, in] :: JSON string of residual model type and params

```
void SETMIXTUREcpp (Handle &model_hdl, const std::string &hMixNme, std::vector<
double > &z, int &ierr, const bool reuse_default_handle=false, const std::string
&ideal_gas_model_params=R"({ "model_type": "}" + kDefaultIdealGasModelType +
R"("})", const std::string &residual_model_params=R"({ "model_type": "}" +
kDefaultResidualModelType + R"("}"))
```

Call the SETMIX routine for a predefined mixture without the need to pass hFmix, hrf, ncc, hf, or herr. It is not necessary to declare the length of hMixNme as 255 bytes long. A path can be included if needed. The extension “.mix” is not required. If it is necessary to set the reference state, call subroutine FLAGS first. The composition of the mixture will be returned in the z array. If ierr comes back non-zero, call the ERRMSG routine to obtain it.

Parameters

- **model_hdl** – [uint64_t&, out] :: Model handle
- **hMixNme** – [const string&, in] :: String of any character length containing the mixture file name
- **z** – [std::vector<double>&, out] :: Composition array (mole fractions), maximum of 20 elements
- **ierr** – [int&, out] :: Error flag
- **reuse_default_handle** – [const bool, in] :: Reuse default, legacy model handle (default: false)
- **ideal_gas_model_params** – [const string&, in] :: JSON string of ideal gas model type and params

- **residual_model_params** – [const string&, in] :: JSON string of residual model type and params

```
void SETMODcpp(const Handle model_hdl, const int ncomp, const std::string &htype, const  
               std::string &hmix, const std::string &hcomp, int &ierr, std::string &herr, const int  
               htype_length = 3, const int hmix_length = 3, const int hcomp_length = 60, const int  
               herr_length = 255)
```

Set model(s) other than the NIST-recommended ('NBS') ones.

This subroutine must be called before SETUP; it need not be called at all if the default (NIST-recommended) models are desired.

Flags flags

EOS Equation of state

ETA Viscosity

TCX Thermal conductivity

STN Surface tension

NBS Reset all of the above model types to 'NBS' (values of hmix and hcomp are ignored)

Flags flags

NBS Use NIST recommendation for specified fluid/mixture

HMX Mixture Helmholtz model for thermodynamic properties

ECS Extended corresponding states for viscosity or thermal conductivity

STX Surface tension mixture model

Flags flags

NBS NIST recommendation for specified fluid/mixture

FEQ Helmholtz energy model

BWR Pure fluid modified Benedict-Webb-Rubin (MBWR)

ECS Pure fluid thermo extended corresponding states / extended corresponding states (all fluids)

PRT Peng-Robinson (PRT model from fluid file)

VS1 The 'composite' model for R134a, R152a, NH3, etc.

VS2 Younglove-Ely model for hydrocarbons

VS4 Generalized friction theory of Quinones-Cisneros and Deiters

VS5 Chung et al. (1988) predictive model

VS6 Vesovic form of VS1 model

VS7 Polynomial/exponential model
TC1 The ‘composite’ model for R134a, R152a, etc.
TC2 Younglove-Ely model for hydrocarbons
TC5 Chung et al. (1988) predictive model
ST1 Surface tension as $f(\tau)$; $\tau = 1 - T/T_c$

Flags flags

0 Successful **113** ncomp outside of bounds

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **ncomp** – [const int, in] :: Number of components (1 for pure fluid)
- **htype** – [const std::string&, in] :: Flag indicating which model to set (character*3)
- **hmix** – [const std::string&, in] :: Mixture model for the property specified in htype (character*3); ignored if number of components = 1
- **hcomp** – [const std::string&, in] :: Component model(s) for property specified in htype; a concatenated string of up to 20 three-character codes, one per component
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **htype_length** – [const int, in] :: length of variable htype (default: 3)
- **hmix_length** – [const int, in] :: length of variable hmix (default: 3)
- **hcomp_length** – [const int, in] :: length of variable hcomp (default: 60)
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **SETNCcpp**(const *Handle* model_hdl, const int ncomp)

Allow the user to modify the value of nc (the number of components in a mixture) so that a subset of the loaded mixture can be used.

For example, a 4 component mixture could be set up, but nc could be set to 3 to calculate properties of the mixture of the first three components. The last component can then be used as a pure fluid. For example, SETUP could be called to load four fluids, R32.fld, R125.fld, R134a.fld, and R407C.ppf, followed by a call to SETNC(3) so that the 4th component is not used in mixture calculations. The pseudo-pure fluid equation in R407C.ppf can be accessed by calling PUREFLD(4) to get single-phase thermodynamic properties, and VLE states or transport properties that require the first three components can be obtained by calling PUREFLD(0) and setting the composition array z with the appropriate values.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle

- **ncomp** – [const int, in] :: Number of components in the mixture

void **SETPATHcpp**(const std::string &hpth, const int hpth_length = 255)

Set the path where the fluid files are located.

Parameters

- **hpth** – [const string&, in] :: Location of the fluid files. The path does not need to contain the ending “/” and it can point directly to the location of the files or to a directory containing a fluids subdirectory (with the corresponding fluid files), named either ‘fluids’ or ‘FLUIDS’ for example, hpth=’C:/Program Files (x86)/Refprop’
- **hpth_length** – [const int, optional] :: length of variable hpth (default: 255)

void **SETREFDIRcpp**(const *Handle* model_hdl, const std::string &hpth, const int hpth_length = 255)

Set a path to the location of original fluid files so that a user can specify where their fluid file is located, but the reference files needed for transport properties, etc., such as nitrogen.fld, can be found.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **hpth** – [const std::string&, in] :: Location of the fluid files (character*255). The path does not need to contain the ending “”. For example: hpth=’C:\Program Files\Refprop\fluids’
- **hpth_length** – [const int, in] :: length of variable hpth (default: 255)

void **SETREFcpp**(const *Handle* model_hdl, const std::string &hrf, const int ixflag, const std::vector<double> &x0, const double h0, const double s0, const double T0, const double P0, int &ierr, std::string &herr, const int hrf_length = 3, const int herr_length = 255)

Set reference state enthalpy and entropy.

This function must be called after SETUPcpp; it need not be called at all if the reference state specified in the call to SETUPcpp is to be used.

Flags flags

NBP h,s = 0 at normal boiling point(s)

ASH h,s = 0 for saturated liquid at -40 C (ASHRAE convention)

IIR h = 200 kJ/kg, s = 1 kJ/kg-K for sat. liquid at 0 C (IIR convention)

DEF Default reference state as specified in fluid file

OTH Other, as specified by h0, s0, T0, P0 (real gas state)

OT0 Other, as specified by h0, s0, T0, P0 (ideal-gas state)

NA Not applicable, do not set up the reference state. Do not use except for EOS testing.

****'???'**** Set hrf to the value of the current reference state and exit

Flags flags

1 Reference state applied to pure components **2** Reference state applied to mixture x0

Flags flags

0 Successful **22** Tmin > Tref for IIR reference state **23** Tcrit < Tref for IIR reference state
24 Tmin > Tref for ASHRAE reference state **25** Tcrit < Tref for ASHRAE reference state
26 Tmin > Tnbp for NBP reference state **-28** Can't apply 'DEF' to mixture; will apply to pure components **-29** Unknown reference state specified; will use 'DEF' **119** Convergence failure in calculating reference state

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **hrf** – [const std::string&, in] :: Reference state for thermodynamic calculations (character*3)
- **ixflag** – [const int, in] :: Composition flag
- **x0** – [const std::vector<double>&, in] :: Composition for which h0 and s0 apply (array of mole fractions). This is useful for mixtures of a predefined composition, e.g., refrigerant blends such as R410A. Only has meaning if ixflag = 2.
- **h0** – [const double, in] :: Reference state enthalpy at T0, P0, and x0 [J/mol] (only has meaning if hrf = 'OTH' or 'OT0')
- **s0** – [const double, in] :: Reference state entropy at T0, P0, and x0 [J/mol-K] (only has meaning if hrf = 'OTH' or 'OT0')
- **T0** – [const double, in] :: Reference state temperature [K] (only has meaning if hrf = 'OTH' or 'OT0'). T0 = -1 indicates saturated liquid at normal boiling point (bubble point for a mixture)
- **P0** – [const double, in] :: Reference state pressure [kPa] (only has meaning if hrf = 'OTH' or 'OT0'). P0 = -1 indicates saturated liquid at T0 (and x0). P0 = -2 indicates saturated vapor at T0 (and x0)
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **hrf_length** – [const int, in] :: length of variable hrf (default: 3)
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void SETUPcpp (Handle &model_hdl, const int ncomp, const std::string &hFiles,
const std::string &hFmix, const std::string &hrf, int &ierr, std::string &herr,
const bool reuse_default_handle=false, const std::string
&ideal_gas_model_params=R"({ "model_type": "}" + kDefaultIdealGasModelType +
R"(" })", const std::string &residual_model_params=R"({ "model_type": "}" +
kDefaultResidualModelType + R"(" }"))
```

Define models and initialize arrays.

Flags flags

- DEF** Default reference state as specified in fluid file
- NBP** $h, s = 0$ at pure component normal boiling point
- ASH** $h, s = 0$ for sat. liquid at -40 C (ASHRAE convention)
- IIR** $h = 200$ kJ/kg and $s = 1$ kJ/kg-K for sat. liquid at 0 C (IIR convention)

Flags flags

0 Successful **101** Error in opening file **102** Error in file or premature end of file **-107** Unknown model encountered in file **105** Specified model not found **-105** Must use routine SETREF for (OTH) reference state choice **111** Error in opening mixture file **112** Mixture file of wrong type **114** nc not equal to the nc sent to SETMOD **-117** Binary pair not found, all parameters will be estimated **117** Mixture parameters not available, mixture is outside the range of the model and calculations

will not be made

Parameters

- model_hdl** – [const uint64_t, in] :: Model handle
- ncomp** – [const int, in] :: Number of components (1 for pure fluid) (integer). If called with ncomp=-1, the version number will be returned in ierr.
- hFiles** – [const std::string&, in] :: Delimited file names specifying the fluid or the mixture components; e.g., 'fluids\r134a.fld' (DOS) ':fluids:r134a.fld' (Mac) '[full path]/fluids/r134a.fld' (UNIX)
- hFmix** – [const std::string&, in] :: Name of file containing mixture coefficients; e.g., 'fluids\HMX.BNC'
- hrf** – [const std::string&, in] :: Reference state for thermodynamic calculations (character*3). Other choices are possible, see SETREFcpp
- ierr** – [int&, out] :: Error flag
- herr** – [std::string&, out] :: Error string
- reuse_default_handle** – [const bool, in] :: Reuse default, legacy model handle (default: false)
- ideal_gas_model_params** – [const string&, in] :: JSON string of ideal gas model type and params
- residual_model_params** – [const string&, in] :: JSON string of residual model type and params

void **SPLNVALcpp**(const *Handle* model_hdl, const int isp, const int iderv, const double a, double &f, int &ierr, std::string &herr, const int herr_length = 255)

Calculates the value of a spline or the derivative of the spline at the specified value.

Flags flags

0 Molar density of the known phase **1 to nc** Composition of the incipient phase **nc+1** Temperature **nc+2** Pressure **nc+3** Molar density of the equilibrium phase **nc+4** Enthalpy **nc+5** Entropy

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **isp** – [const int, in] :: Indicator for which spline to use
- **iderv** – [const int, in] :: Values of -1 and -2 return lowest and highest density values of the splines, a value of 0 returns spline function value, a value of 1 returns the derivative of the spline with respect to the input value, and a value of 2 returns the 2nd derivative.
- **a** – [const double, in] :: Input value (molar density of the known phase) [mol/L]
- **f** – [double&, out] :: Desired output value
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **SPLNROOTcpp**(const *Handle* model_hdl, const int isp, const int iderv, double &f, double &a, int &ierr, std::string &herr, const int herr_length = 255)

Calculates the root of a given value of a spline function.

Flags flags

1 to nc Composition **nc+1** Temperature **nc+2** Pressure **nc+3** Density **nc+4** Enthalpy or Energy (depending on the value of ieflag) **nc+5** Entropy

Flags flags

-1 Returns the lower root value **-2** Returns the upper root value **0** Returns the spline root (density) that equals f **1** Returns the root (density) where the derivative of the spline with respect to D equals f (set f=0 to find maximum or minimum)

Flags flags

0 Successful **151** Routine did not converge.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **isp** – [const int, in] :: Indicator for which spline to use
- **iderv** – [const int, in] :: Indicator for which value to return
- **f** – [double&, in] :: Value of spline function

- **a** – [double&, inout] :: Root value (initial value required since some splines can be double valued)
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void STNcpp(const Handle model_hdl, const double T, const double Dl, const double Dv, const  
std::vector<double> &x, const std::vector<double> &y, double &sigma, int &ierr,  
std::string &herr, const int herr_length = 255)
```

Compute surface tension with appropriate core model. For mixtures, this routine requires that the saturation densities and vapor compositions be sent as inputs. If these are not available, call SURFTcpp.

The critical temperature used is that of the current equation of state. This may differ slightly from that used in the original correlation of the surface tension; this change is necessary to give proper behavior of surface tension near the critical point and to avoid possible numerical crashes.

Flags flags

0 Successful **1** $T > T_{crit}$ **502** Unknown model **151** Failed to converge

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **Dl** – [const double, in] :: Molar density of the liquid phase [mol/L]
- **Dv** – [const double, in] :: Molar density of the vapor phase [mol/L]
- **x** – [const std::vector<double>&, in] :: Composition of liquid phase (vector of mole fractions), maximum of 20 elements
- **y** – [const std::vector<double>&, in] :: Composition of vapor phase (vector of mole fractions), maximum of 20 elements
- **sigma** – [double&, out] :: Surface tension [N/m]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string (character*255)
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void SURFTcpp(const Handle model_hdl, const double T, double &Dl, const std::vector<double> &z,  
double &sigma, int &ierr, std::string &herr, const int herr_length = 255)
```

Compute surface tension as a function of T. SATT is called to obtain the liquid density. If this is already known then your calling routines should use subroutine STNcpp to greatly reduce the time needed in the calculation of the surface tension.

Flags flags

0 Successful Other error messages returned from SATT or STN

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **Dl** – [double&, out] :: Molar density of the liquid phase [mol/L] (only returned for mixtures)
- **z** – [const std::vector<double>&, in] :: Composition of the liquid phase (vector of mole fractions), maximum of 20 elements
- **sigma** – [double&, out] :: Surface tension [N/m]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string (character*255)
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void SURTENCcpp(const Handle model_hdl, const double T, const double Dl, const double Dv, const
std::vector<double> &x, const std::vector<double> &y, double &sigma, int &ierr,
std::string &herr, const int herr_length = 255)
```

With version 10 of Refprop, this routine should no longer be used, and STN or SURFT should be used instead. (See subroutine STN for the description of all variables.)

Parameters

- **model_hdl** – [const uint64_t, in] :: XXXXXXXXXXXX
- **T** – [const double, in] :: XXXXXXXXXXXX
- **Dl** – [const double, in] :: XXXXXXXXXXXX
- **Dv** – [const double, in] :: XXXXXXXXXXXX
- **x** – [const std::vector<double>&, in] :: XXXXXXXXXXXX
- **y** – [const std::vector<double>&, in] :: XXXXXXXXXXXX
- **sigma** – [double&, out] :: XXXXXXXXXXXX
- **ierr** – [int&, out] :: XXXXXXXXXXXX
- **herr** – [std::string&, out] :: XXXXXXXXXXXX
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void TDFLSHcpp(const Handle model_hdl, const double T, const double D, const
std::vector<double> &z, double &P, double &Dl, double &Dv, std::vector<double>
&x, std::vector<double> &y, double &q, double &e, double &h, double &s, double
&Cv, double &Cp, double &w, int &ierr, std::string &herr, const int herr_length =
255)
```

Flash calculation given temperature, bulk density, and bulk composition.

This routine accepts both single-phase and two-phase states as inputs; for single-phase calculations, the subroutine THERM is much faster.

Flags flags

0 Successful **213** Density calculation did not converge **215** Supercritical density calculation did not converge **223** Bubble point did not converge **224** Dew point did not converge

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Density [mol/L]
- **z** – [const std::vector<double>&, in] :: Bulk composition (vector of mole fractions), maximum of 20 elements
- **P** – [double&, out] :: Pressure [kPa]
- **Dl** – [double&, out] :: Molar density of the liquid phase [mol/L]
- **Dv** – [double&, out] :: Molar density of the vapor phase [mol/L]
- **x** – [std::vector<double>&, out] :: Composition of the liquid phase (vector of mole or mass fractions)
- **y** – [std::vector<double>&, out] :: Composition of the vapor phase (vector of mole or mass fractions)
- **q** – [double&, out] :: Vapor quality (mol/mol)
- **e** – [double&, out] :: Internal energy [J/mol]
- **h** – [double&, out] :: Enthalpy [J/mol]
- **s** – [double&, out] :: Entropy [J/mol-K]
- **Cv** – [double&, out] :: Isochoric heat capacity [J/mol-K]
- **Cp** – [double&, out] :: Isobaric heat capacity [J/mol-K]
- **w** – [double&, out] :: Speed of sound [m/s]
- **ierr** – [int&, out] :: Error code
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void TEFL1cpp(const Handle model_hdl, const double T, const double e, const std::vector<double>
    &z, const double Dmin, const double Dmax, double &D, int &ierr, std::string &herr,
    const int herr_length = 255)
```

Iterate for single-phase density as a function of temperature, energy, and composition.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **e** – [const double, in] :: Internal energy [J/mol]
- **z** – [const std::vector<double>&, in] :: Bulk composition (vector of mole fractions), maximum of 20 elements
- **Dmin** – [const double, in] :: Lower bound on density [mol/L]
- **Dmax** – [const double, in] :: Upper bound on density [mol/L]
- **D** – [double&, out] :: Density [mol/L]
- **ierr** – [int&, out] :: Error code
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void THERMcpp(const Handle model_hdl, const double T, const double D, const std::vector<double>
    &z, double &P, double &e, double &h, double &s, double &Cv, double &Cp, double
    &w, double &hjt)
```

Compute thermal quantities as a function of temperature, density, and composition from core functions (Helmholtz energy, ideal gas heat capacity and various derivatives and integrals).

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **P** – [double&, out] :: Pressure [kPa]
- **e** – [double&, out] :: Internal energy [J/mol]
- **h** – [double&, out] :: Enthalpy [J/mol]
- **s** – [double&, out] :: Entropy [J/mol-K]
- **Cv** – [double&, out] :: Isochoric heat capacity [J/mol-K]
- **Cp** – [double&, out] :: Isobaric heat capacity [J/mol-K]
- **w** – [double&, out] :: Speed of sound [m/s]
- **hjt** – [double&, out] :: Isenthalpic Joule-Thomson coefficient [K/kPa]

```
void THERM0cpp(const Handle model_hdl, const double T, const double D, const
               std::vector<double> &z, double &P0, double &e0, double &h0, double &s0,
               double &Cv0, double &Cp00, double &w0, double &a0, double &g0)
```

Compute ideal-gas thermal quantities as a function of temperature, density, and composition from core functions. This routine is the same as THERM, except it only calculates ideal gas properties ($Z=1$) at any temperature and density.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **P0** – [double&, out] :: Pressure [kPa]
- **e0** – [double&, out] :: Internal energy [J/mol]
- **h0** – [double&, out] :: Enthalpy [J/mol]
- **s0** – [double&, out] :: Entropy [J/mol-K]
- **Cv0** – [double&, out] :: Isochoric heat capacity [J/mol-K]
- **Cp00** – [double&, out] :: Isobaric heat capacity [J/mol-K]
- **w0** – [double&, out] :: Speed of sound [m/s]
- **a0** – [double&, out] :: Helmholtz energy [J/mol]
- **g0** – [double&, out] :: Gibbs free energy [J/mol]

```
void THERM2cpp(const Handle model_hdl, const double T, const double D, const
               std::vector<double> &z, double &P, double &e, double &h, double &s, double
               &Cv, double &Cp, double &w, double &zz, double &hjt, double &a, double &g,
               double &xkappa, double &beta, double &dPdD, double &d2PdD2, double &dPdT,
               double &dDdT, double &dDdP, double &d2PdT2, double &d2PdTD, double
               &spare3, double &spare4)
```

Compute thermal quantities as a function of temperature, density, and composition from core functions (Helmholtz energy, ideal gas heat capacity, and various derivatives and integrals). See warning in subroutines THERM or ALLPROPS.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements

- **P** – [double&, out] :: Pressure [kPa]
- **e** – [double&, out] :: Internal energy [J/mol]
- **h** – [double&, out] :: Enthalpy [J/mol]
- **s** – [double&, out] :: Entropy [J/mol-K]
- **g** – [double&, out] :: Gibbs free energy [J/mol] Compute thermal quantities as a function of temperature, density, and composition. This routine is the simply the combination of several others. See warning in subroutines THERM or ALLPROPS.
- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **P** – [double&, out] :: Pressure [kPa]
- **e** – [double&, out] :: Internal energy [J/mol]
- **h** – [double&, out] :: Enthalpy [J/mol]
- **s** – [double&, out] :: Entropy [J/mol-K]
- **Cv** – [double&, out] :: Isochoric heat capacity [J/mol-K]
- **Cp** – [double&, out] :: Isobaric heat capacity [J/mol-K]
- **w** – [double&, out] :: Speed of sound [m/s]
- **zz** – [double&, out] :: Compressibility factor [-]
- **hjt** – [double&, out] :: Isenthalpic Joule-Thomson coefficient [K/kPa]
- **a** – [double&, out] :: Helmholtz energy [J/mol]
- **g** – [double&, out] :: Gibbs free energy [J/mol]
- **xkappa** – [double&, out] :: Isothermal compressibility [1/kPa]
- **beta** – [double&, out] :: Volumetric expansivity [1/K]
- **dPdD** – [double&, out] :: 1st derivative of pressure with respect to density at constant temperature [kPa-m³/mol]
- **d2PdD2** – [double&, out] :: 2nd derivative of pressure with respect to density at constant temperature [kPa-m⁶/mol²]
- **dPdT** – [double&, out] :: 1st derivative of pressure with respect to temperature at constant density [Pa/K]
- **dDdT** – [double&, out] :: 1st derivative of density with respect to temperature at constant pressure [mol/(m³-K)]

- **dDdP** – [double&, out] :: 1st derivative of density with respect to pressure at constant temperature [mol/(m³-kPa)]
- **d2PdT2** – [double&, out] :: 2nd derivative of pressure with respect to temperature at constant density [kPa/K²]
- **d2PdTD** – [double&, out] :: 2nd derivative of pressure with respect to density and temperature [kPa-m³/K-mol]
- **spare3** – [double&, out] :: Spare 3 [-]
- **spare4** – [double&, out] :: Spare 4 [-]

void **THERM3cpp**(const *Handle* model_hdl, const double T, const double D, const std::vector<double> &z, double &xkappa, double &beta, double &xisenk, double &xkt, double &betas, double &bs, double &xkkt, double &thrott, double &pi, double &spht)

Compute miscellaneous thermodynamic properties. See warning in subroutines THERM or ALLPROPS.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **xkappa** – [double&, out] :: Isothermal compressibility [1/kPa]
- **beta** – [double&, out] :: Volumetric expansivity [1/K]
- **xisenk** – [double&, out] :: Isentropic expansion coefficient [-]
- **xkt** – [double&, out] :: Isothermal expansion coefficient [-]
- **betas** – [double&, out] :: Adiabatic compressibility [1/kPa]
- **bs** – [double&, out] :: Adiabatic bulk modulus [kPa]
- **xkkt** – [double&, out] :: Isothermal bulk modulus [kPa]
- **thrott** – [double&, out] :: Isothermal throttling coefficient [L/mol]
- **pi** – [double&, out] :: Internal pressure [kPa]
- **spht** – [double&, out] :: Specific heat input [J/mol]

void **THFL1cpp**(const *Handle* model_hdl, const double T, const double h, const std::vector<double> &z, const double Dmin, const double Dmax, double &D, int &ierr, std::string &herr, const int herr_length = 255)

Iterate for single-phase density as a function of temperature, enthalpy, and composition.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle

- **T** – [const double, in] :: Temperature [K]
- **h** – [const double, in] :: Enthalpy [J/mol]
- **z** – [const std::vector<double>&, in] :: Bulk composition (vector of mole fractions), maximum of 20 elements
- **Dmin** – [const double, in] :: Lower bound on density [mol/L]
- **Dmax** – [const double, in] :: Upper bound on density [mol/L]
- **D** – [double&, out] :: Density [mol/L]
- **ierr** – [int&, out] :: Error code
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void TPFL2cpp(const Handle model_hdl, const double T, const double P, const std::vector<double>
    &z, double &Dl, double &Dv, std::vector<double> &x, std::vector<double> &y,
    double &q, int &ierr, std::string &herr, const int herr_length = 255)
```

Flash calculation given temperature, pressure, and bulk composition.

This routine accepts only two-phase states as inputs; if the phase is not known use TPFLSH. Use TPRHO for single-phase states.

Flags flags

0 Successful **213** TPRHO did not converge **226** Iteration did not converge

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **P** – [const double, in] :: Pressure [kPa]
- **z** – [const std::vector<double>&, in] :: Overall composition (vector of mole fractions), maximum of 20 elements
- **Dl** – [double&, in\out] :: Molar density of the liquid phase, initial guess on input [mol/L]
- **Dv** – [double&, in\out] :: Molar density of the vapor phase, initial guess on input [mol/L]
- **x** – [std::vector<double>&, in\out] :: Composition of the liquid phase, initial guess on input (vector of mole fractions)
- **y** – [std::vector<double>&, in\out] :: Composition of the vapor phase, initial guess on input (vector of mole fractions)
- **q** – [double&, in\out] :: Vapor quality on a MOLAR basis, initial guess on input (moles vapor/total moles)

- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void TPFLSHcpp(const Handle model_hdl, const double T, const double P, const std::vector<double>
    &z, double &D, double &Dl, double &Dv, std::vector<double> &x,
    std::vector<double> &y, double &q, double &e, double &h, double &s, double
    &Cv, double &Cp, double &w, int &ierr, std::string &herr, const int herr_length =
    255)
```

Flash calculation given temperature, pressure, and bulk composition.

This routine accepts both single-phase and two-phase states as inputs; for single-phase calculations, the subroutine TPRHO is much faster.

Flags flags

0 Successful **213** Density calculation did not converge **215** Supercritical density calculation did not converge **223** Bubble point did not converge **224** Dew point did not converge

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **P** – [const double, in] :: Pressure [kPa]
- **z** – [const std::vector<double>&, in] :: Overall composition (vector of mole fractions), maximum of 20 elements
- **D** – [double&, out] :: Density [mol/L]
- **Dl** – [double&, out] :: Molar density of the liquid phase [mol/L]
- **Dv** – [double&, out] :: Molar density of the vapor phase [mol/L]
- **x** – [std::vector<double>&, out] :: Composition of the liquid phase (vector of mole or mass fractions)
- **y** – [std::vector<double>&, out] :: Composition of the vapor phase (vector of mole or mass fractions)
- **q** – [double&, out] :: Vapor quality (mol/mol)
- **e** – [double&, out] :: Internal energy [J/mol]
- **h** – [double&, out] :: Enthalpy [J/mol]
- **s** – [double&, out] :: Entropy [J/mol-K]
- **Cv** – [double&, out] :: Isochoric heat capacity [J/mol-K]
- **Cp** – [double&, out] :: Isobaric heat capacity [J/mol-K]
- **w** – [double&, out] :: Speed of sound [m/s]

- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void TPRH0cpp(const Handle model_hdl, const double T, const double P, const std::vector<double>
    &z, const int kph, const int kguess, double &D, int &ierr, std::string &herr, const int
    herr_length = 255)
```

Iterate for density as a function of temperature, pressure, and composition of a specified phase.

Invalid densities will be returned for T and P outside the range of validity, e.g., $P > P_{\text{melt}}$, $P < P_{\text{sat}}$ for $kph=1$, etc.

Flags flags

1 Liquid phase **2** Vapor phase **0** Stable phase - NOT ALLOWED (use TPFLSH) (Unless an initial guess is supplied for D) **-1** Force the search in the liquid phase (for metastable points) **-2** Force the search in the vapor phase (for metastable points)

Flags flags

0 No first guess for D provided **1** First guess for D provided

Flags flags

0 Successful **201** Illegal input ($kph \leq 0$) **202** Liquid-phase iteration did not converge **203** Vapor-phase iteration did not converge

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **P** – [const double, in] :: Pressure [kPa]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **kph** – [const int, in] :: Phase flag
- **kguess** – [const int, in] :: Guess flag
- **D** – [double&, in\out] :: Molar density, initial guess on input if kguess = 1 [mol/L]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

```
void TPRHOPRcpp(const Handle model_hdl, const double T, const double P, const
    std::vector<double> &z, double &D1, double &D2)
```

Compute density with a volume-translated modification of the Peng-Robinson equation of state:

$$P = RT/(v+c+b) - a/((v+c)*(v+c+b)+b*(v+c+b))$$

c is a translation constant, as given in Peneloux and Rauzy, Fluid Phase Equilib. 8:7-23, 1982.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **P** – [const double, in] :: Pressure [kPa]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **D1** – [double&, out] :: Largest density root [mol/L]
- **D2** – [double&, out] :: Smallest density root [mol/L]

```
void TRNPRPcpp(const Handle model_hdl, const double T, const double D, const
               std::vector<double> &z, double &eta, double &tcx, int &ierr, std::string &herr,
               const int herr_length = 255)
```

Compute the transport properties thermal conductivity and viscosity as functions of temperature, density, and composition.

If $q > 0$ and $q < 1$, then values of the liquid compositions will be returned in the x and y arrays, and the transport properties of the liquid and vapor states can be calculated as follows:

```
TRNPRP (model_hdl, T, D1, x, etaliq, tcxliq, ierr, herr)
TRNPRP (model_hdl, T, Dv, y, etavap, tcxvap, ierr, herr)
```

Flags flags

0 Successful **502** Unknown viscosity or thermal conductivity model specified **73** One or more inputs are out of bounds for the ECS model **74** Inputs to the vis. and th. cond. correlations are out of range **540** Transport equations are not available for one or more of the fluids **541** Transport equations are not available for mixtures with water at molar concentrations greater than 5% **542** Transport equations are not available for mixtures with alcohols **543** Transport equations are not available for the ammonia/water mixture **-508** Invalid region for viscosity of reference fluid 1 **-558** 2-D Newton-Raphson method for conformal temperature and density did not converge **-560** Pure fluid is exactly at the critical point; thermal conductivity is infinite **561** Pure fluid correlation produced an erroneous value for either viscosity or thermal conductivity **5##** Pure fluid correlation out of range; attempted to use ECS method

Warning

Do NOT call this routine for two-phase states, otherwise it will return a metastable state if near the phase boundary or complete nonsense at other conditions. The value of q returned from the flash routines will indicate a two phase state by returning a value between 0 and 1. In such a situation, the transport properties can only be calculated for the saturated liquid and vapor states. For example, when calling PHFLSH:

```
PHFLSH (model_hdl, P, h, z, T, D, D1, Dv, x, y, q, e, s, Cv, Cp, w, ierr, herr)
```

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **D** – [const double, in] :: Molar density [mol/L]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **eta** – [double&, out] :: Viscosity [uPa·s]
- **tcx** – [double&, out] :: Thermal conductivity [W/m-K]
- **ierr** – [int&, out] :: Error flag
- **herr** – [std::string&, out] :: Error string (character*255)
- **herr_length** – [const int, in] :: length of variable herr (default: 255)

void **UNSETAGAcpp**(const *Handle* model_hdl)

Load original values into arrays changed in the call to SETAGA. This routine resets the values back to those loaded when SETUP was called.

Parameters

model_hdl – [const uint64_t, in] :: Model handle

void **UNSETFLUIDScpp**(const *Handle* model_hdl)

Removes the model and deallocates the memory

Parameters

model_hdl – [const uint64_t, in] :: Model handle

void **UNSETPATHcpp**()

Unsets the path where fluid files are located, stopping loading of files unless a path is specified in the name.

void **VIRBcpp**(const *Handle* model_hdl, const double T, const std::vector<double> &z, double &B)

Compute the second virial coefficient B (L/mol) as a function of temperature T (K) and composition x (vector of mole fractions). This routine approximates B . For pure fluids, the routine VIRBCD is exact.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **B** – [double&, out] :: Second virial coefficient [L/mol]

void **VIRBAcpp**(const *Handle* model_hdl, const double T, const std::vector<double> &z, double &Ba)

Compute the second acoustic virial coefficient Ba (L/mol) as a function of temperature (K) and composition x (vector of mole fractions). For further information, see Trusler and Zarari, J. Chem. Thermodyn., 28:329-335,

- Gillis and Moldover, Int. J. Thermophys., 17(6):1305-1324, 1996. This routine approximates Ba. For pure fluids, the routine VIRBCD is exact. The following equations are used in the calculations:

$$\gamma_{ig} = \frac{c_{pig}}{c_{pig} - R}$$

$$Ba = 2B + 2(\gamma_{ig} - 1)T \frac{dB}{dT} + \frac{(\gamma_{ig} - 1)^2}{\gamma_{ig}} T^2 \frac{d^2B}{dT^2}$$

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **Ba** – [double&, out] :: Second acoustic virial coefficient [L/mol]

void **VIRBCDcpp**(const *Handle* model_hdl, const double T, const std::vector<double> &z, double &B, double &C, double &D, double &E)

Compute virial coefficients as a function of temperature and composition. The routine currently works only for pure fluids and for the Helmholtz equation. All values are computed exactly based on the terms in the EOS, not as was done in VIRB by calculating properties at a density of 1d-8.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **B** – [double&, out] :: Second virial coefficient [L/mol] = a01

- **C** – [double&, out] :: Third virial coefficient $[(L/mol)^2] = a02$
- **D** – [double&, out] :: Fourth virial coefficient $[(L/mol)^3] = a03/2d0$
- **E** – [double&, out] :: Fifth virial coefficient $[(L/mol)^4] = a04/6d0$ assume nomenclature of $a02=[\partial^2(\alpha p)/\partial(\delta)^2]$ above

void **VIRCAcpp**(const *Handle* model_hdl, const double T, const std::vector<double> &z, double &Ca)

Compute the third acoustic virial coefficient $Ca (L/mol)^2$ as a function of temperature (K) and composition x (vector of mole fractions). For further information, see Estela-Urbe and Trusler, Int. J. Thermophys., 21(5):1033,

- Gillis and Moldover, Int. J. Thermophys., 17(6):1305-1324, 1996. This routine approximates Ca . For pure fluids, the routine VIRBCD is exact. The following equations are used in the calculations:

$$\gamma_{ig} = \frac{c_{pig}}{c_{pig} - R}$$

$$q = B + (2\gamma_{ig} - 1)T \frac{dB}{dT} + (\gamma_{ig} - 1)T^2 \frac{d^2B}{dT^2}$$

$$Ca = (\gamma_{ig} - 1)q^2 + (2\gamma_{ig} + 1)C$$

$$Ca = \frac{Ca + (\gamma_{ig}^2 - 1)T \frac{dC}{dT} + \frac{(\gamma_{ig} - 1)^2}{2} T^2 \frac{d^2C}{dT^2}}{\gamma_{ig}}$$

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle
- **T** – [const double, in] :: Temperature [K]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **Ca** – [double&, out] :: Third acoustic virial coefficient $[(L/mol)^2]$

void **VIRCCpp**(const *Handle* model_hdl, const double T, const std::vector<double> &z, double &C)

Compute the third virial coefficient $C (L/mol)^2$ as a function of temperature T (K) and composition x (array of mole fractions). This routine approximates C . For pure fluids, the routine VIRBCD is exact.

Parameters

- **model_hdl** – [const uint64_t, in] :: Model handle

- **T** – [const double, in] :: Temperature [K]
- **z** – [const std::vector<double>&, in] :: Composition (vector of mole fractions), maximum of 20 elements
- **C** – [double&, out] :: Third virial coefficient $[(L/mol)^2] = a02$

Variables

```
const std::string kDefaultIdealGasModelType = "IdealHelmholtz"
```

```
const std::string kDefaultResidualModelType = "multifluid"
```

INDICES AND TABLES

H

Handle (C++ type), 5

I

IRefprop (C++ class), 5

IRefprop::~IRefprop (C++ function), 5

IRefprop::cas (C++ function), 5

IRefprop::name (C++ function), 5

R

RefpropManager (C++ class), 5

RefpropManager::create_model (C++ function), 5

RefpropManager::get_model (C++ function), 5

RefpropManager::Handle (C++ type), 5

rp (C++ type), 5

rp::A_ideal_gas (C++ function), 6

rp::A_residual (C++ function), 6

rp::AGcpp (C++ function), 6

rp::B12cpp (C++ function), 7

rp::BLCRVcpp (C++ function), 7

rp::CHEMPOTcpp (C++ function), 7

rp::CP0cpp (C++ function), 8

rp::CRITPcpp (C++ function), 8

rp::CRTPNTcpp (C++ function), 9

rp::CSATKcpp (C++ function), 9

rp::CSTARcpp (C++ function), 10

rp::CV2PKcpp (C++ function), 11

rp::CVCPCpp (C++ function), 11

rp::CVCPCpp (C++ function), 11

rp::DBDTcpp (C++ function), 12

rp::DDDPcpp (C++ function), 12

rp::DDDTcpp (C++ function), 12

rp::DEFL1cpp (C++ function), 13

rp::DERVPVTcpp (C++ function), 13

rp::DHD1cpp (C++ function), 14

rp::DHFL1cpp (C++ function), 14

rp::DIELECcpp (C++ function), 15

rp::DLSATKcpp (C++ function), 15

rp::DPDD2cpp (C++ function), 16

rp::DPDDcpp (C++ function), 16

rp::DPDTcpp (C++ function), 16

rp::DPTSATKcpp (C++ function), 16

rp::DSD1cpp (C++ function), 17

rp::DSFL1cpp (C++ function), 18

rp::DVSATKcpp (C++ function), 18

rp::ENTHALcpp (C++ function), 19

rp::ENTROcpp (C++ function), 19

rp::ERRMSGcpp (C++ function), 19

rp::EXCESScpp (C++ function), 20

rp::FGCTY2cpp (C++ function), 21

rp::FGCTYcpp (C++ function), 20

rp::FLAGScpp (C++ function), 21

rp::FPVcpp (C++ function), 23

rp::FUGCOFcpp (C++ function), 23

rp::GERG04cpp (C++ function), 27

rp::GERG08cpp (C++ function), 28

rp::GETENUMcpp (C++ function), 24

rp::GETFIJcpp (C++ function), 24

rp::GETKTVcpp (C++ function), 25

rp::GETMODcpp (C++ function), 26

rp::GETREFDIRcpp (C++ function), 27

rp::GIBBScpp (C++ function), 28

rp::HEATcpp (C++ function), 29

rp::HEATFRMcpp (C++ function), 29

rp::HMXORDERcpp (C++ function), 30

rp::IDCRVcpp (C++ function), 31

rp::INFOcpp (C++ function), 31

rp::JICRVcpp (C++ function), 32

rp::JTCRVcpp (C++ function), 33

rp::kDefaultIdealGasModelType (C++ member), 90

rp::kDefaultResidualModelType (C++ member), 90

rp::LIMITKcpp (C++ function), 33
rp::LIMITScpp (C++ function), 34
rp::LIMITXcpp (C++ function), 35
rp::LIQSPNDLcpp (C++ function), 36
rp::MASSFLUXcpp (C++ function), 37
rp::MAXPcpp (C++ function), 37
rp::MAXTcpp (C++ function), 38
rp::MELTKcpp (C++ function), 39
rp::MELTPcpp (C++ function), 39
rp::MELTTcpp (C++ function), 40
rp::MLTH20cpp (C++ function), 40
rp::NAMEcpp (C++ function), 40
rp::PASSCMNcpp (C++ function), 41
rp::PDFL1cpp (C++ function), 42
rp::PEFL1cpp (C++ function), 43
rp::PHFL1cpp (C++ function), 43
rp::PHI0cpp (C++ function), 44
rp::PHIDERVcpp (C++ function), 45
rp::PHIHMxcpp (C++ function), 46
rp::PHIKcpp (C++ function), 45
rp::PHIMIXcpp (C++ function), 46
rp::PHIXcpp (C++ function), 47
rp::PRE0Scpp (C++ function), 47
rp::PRESScpp (C++ function), 48
rp::PSATKcpp (C++ function), 48
rp::PSFL1cpp (C++ function), 48
rp::PUREFLDcpp (C++ function), 49
rp::QMASScpp (C++ function), 49
rp::QMOLecpp (C++ function), 50
rp::RDXHMXcpp (C++ function), 51
rp::REDXcpp (C++ function), 52
rp::RESIDUALcpp (C++ function), 52
rp::RIEMcpp (C++ function), 53
rp::RMIX2cpp (C++ function), 53
rp::SATDcpp (C++ function), 53
rp::SATEcpp (C++ function), 54
rp::SATESTcpp (C++ function), 55
rp::SATGUESScpp (C++ function), 56
rp::SATGVcpp (C++ function), 57
rp::SATHcpp (C++ function), 59
rp::SATPcpp (C++ function), 60
rp::SATScpp (C++ function), 61
rp::SATSPLNcpp (C++ function), 62
rp::SATTCpp (C++ function), 62
rp::SATTPcpp (C++ function), 63
rp::SETAGAcpp (C++ function), 65
rp::SETKTVcpp (C++ function), 67
rp::SETMODcpp (C++ function), 70
rp::SETNCcpp (C++ function), 71
rp::SETPATHcpp (C++ function), 72
rp::SETREFcpp (C++ function), 72
rp::SETREFDIRcpp (C++ function), 72
rp::SPLNR00Tcpp (C++ function), 75
rp::SPLNVALcpp (C++ function), 74
rp::STNcpp (C++ function), 76
rp::SUBLPcpp (C++ function), 65
rp::SUBLTcpp (C++ function), 65
rp::SURFTcpp (C++ function), 76
rp::SURTENcpp (C++ function), 77
rp::TDFLSHcpp (C++ function), 77
rp::TEFL1cpp (C++ function), 78
rp::THERM0cpp (C++ function), 79
rp::THERM2cpp (C++ function), 80
rp::THERM3cpp (C++ function), 82
rp::THERMcpp (C++ function), 79
rp::THFL1cpp (C++ function), 82
rp::TPFL2cpp (C++ function), 83
rp::TPFLSHcpp (C++ function), 84
rp::TPRH0cpp (C++ function), 85
rp::TPRHOPRcpp (C++ function), 85
rp::TRNPRPcpp (C++ function), 86
rp::UNSETAGAcpp (C++ function), 87
rp::UNSETFLUIDScpp (C++ function), 87
rp::UNSETPATHcpp (C++ function), 87
rp::VIRBAcpp (C++ function), 88
rp::VIRBCDcpp (C++ function), 88
rp::VIRBcpp (C++ function), 87
rp::VIRCAcpp (C++ function), 89
rp::VIRCcpp (C++ function), 89